

고급머신러닝 보고서

오토인코더

Auto Encoder

전자공학부 전자공학과
2020144005 김윤성
2024. 11. 19 제출

목차 INDEX

01 오토인코더

- PCA
- t-SNE
- 인코더·디코더
- 오토인코더
- 실습 #1)
- 합성곱 오토인코더
- 실습 #2)

02 이상치 탐지

- 이상치 탐지
- 실습 #3)
- 코드 분석
- 실행 결과

03 VAE

- VAE
- MLE
- KLD
- 실습 #4)
- 코드 분석
- 실행 결과

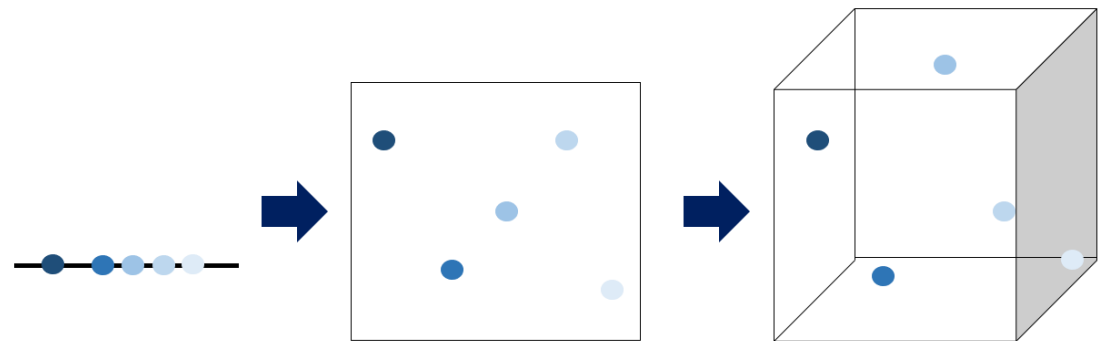
01

오토인코더

Auto Encoder

차원의 저주 The Curse of Dimensionality

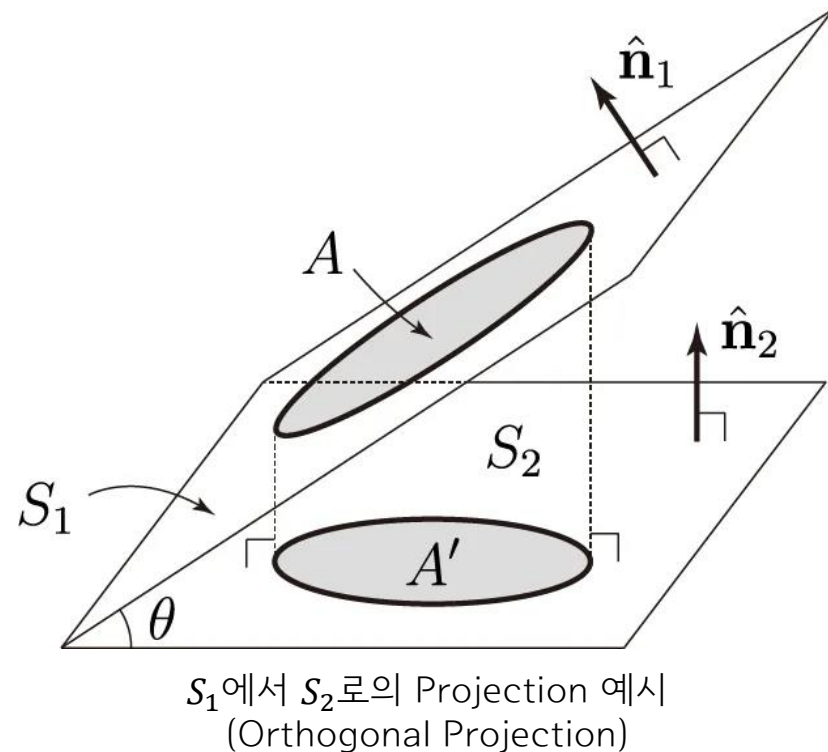
- 데이터 학습을 위해 차원이 증가하면서 학습데이터 수가 차원의 수보다 적어져 성능이 저하되는 현상
- 차원이 증가할 수록 개별 차원 내 학습할 데이터 수가 적어지는(sparse) 현상 발생
- 고차원 공간에서는 데이터가 더 넓게 퍼지기 때문에 밀도가 낮아지고, 결과적으로 머신러닝 모델이 학습에 필요한 충분한 데이터를 확보하기 어려워짐
- 고차원에서 데이터 간 거리(L2 Norm이나 Cosine 유사도 등)는 거의 동일하게 나타나는 경향이 있음
-> 이는 Clustering 이나 Nearest Neighbor의 성능을 저하시킴
- 차원이 높아질수록 모델은 데이터에 대해 Overfitting 될 가능성이 커짐
-> 이는 데이터의 패턴보다 노이즈를 학습하여 일반화 성능이 떨어지게 만듦



차원의 증가에 따른 Sparse 현상

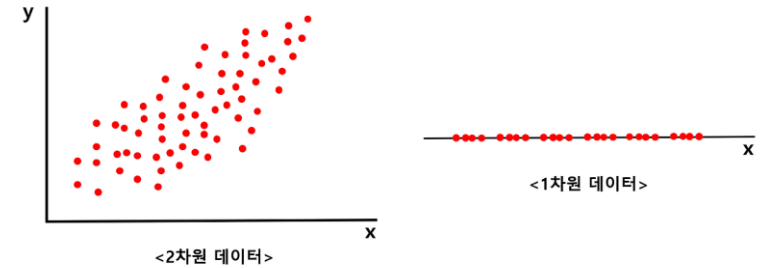
투영 Projection

- 고차원 데이터를 저차원 공간으로 변환하여 데이터의 구조를 단순화하고 분석을 용이하게 만드는 기법
- 고차원 데이터에서는 데이터 포인트 간의 거리가 멀어지고, 모델의 학습이 어려워지는 차원의 저주 현상이 일어나는데, 투영을 통해 차원을 축소하면 이러한 문제를 완화할 수 있음
- 차원을 축소하면 모델의 학습 속도가 빨라지고, 과적합(overfitting) 위험이 감소하여 일반화 성능이 향상될 가능성이 있음
- Linear Mapping, PCA 등이 방법이 존재

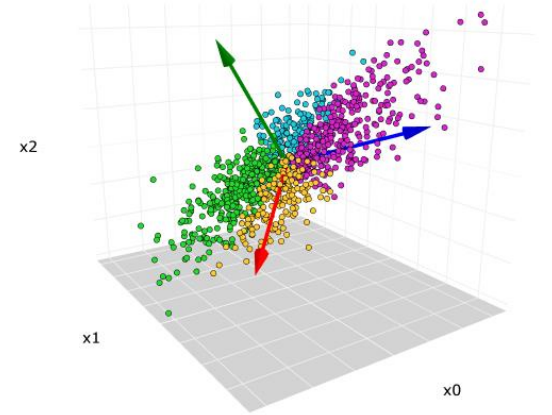


PCA Principal Component Analysis

- 고차원 데이터를 저차원으로 변환하여
데이터의 주요 정보를 효율적으로 표현하는 차원 축소 기법
- 1. 각 변수의 단위와 범위가 다를 경우, 평균을 0으로, 분산을 1로 맞추는
표준화를 수행하여 변수 간의 스케일 차이를 제거
- 2. 표준화된 데이터로부터 변수들 간의 공분산 행렬을 계산하여
변수들 간의 상관관계 파악
- 3. 공분산 행렬의 고유값(데이터의 분산을 설명하는 정도)과
고유벡터(새로운 축, 즉 주성분)를 계산
- 4. 고유값의 크기 순으로 주성분을 정렬하고, 누적 분산 설명력이 높은
상위 주성분을 선택하여 데이터의 차원을 축소
- 5. 선택된 주성분을 기준으로 원본 데이터를 차원 축소



X축으로 단순 투영



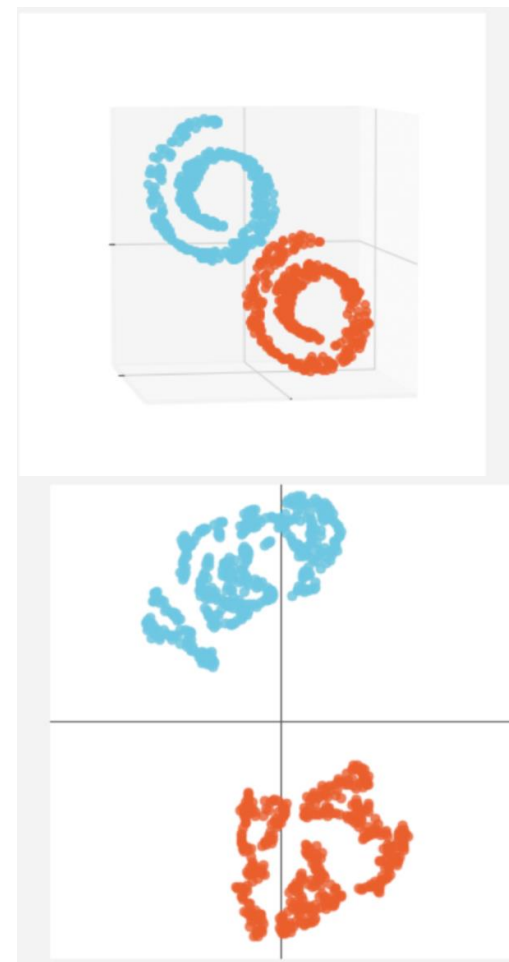
PCA 사용

t-SNE

t-distributed Stochastic Neighbor Embedding

1. 각 데이터 포인트에 대해, 다른 데이터 포인트와의 거리를 기반으로 가우시안 분포를 사용하여 이웃이 될 확률을 계산한다.
즉, 고차원 공간에서 가까운 데이터 포인트일수록 높은 확률을 가진다.
2. 고차원 공간에서 정의한 확률 분포를 저차원 공간에서도 유사하게 재현하기 위해, 저차원 공간에서의 데이터 포인트 간 거리를 기반으로 t-분포*를 사용하여 확률을 계산
3. 고차원 공간과 저차원 공간에서의 확률 분포 간의 차이를 Kullback-Leibler 발산(KL 발산)을 통해 측정하고, 이 값을 최소화하는 방향으로 저차원 공간의 데이터 포인트 위치를 조정

* t-Distribution: 모집단의 분산을 알 수 없고 표본 크기가 작을 때, 모집단의 평균에 대한 추론을 수행하기 위해 사용되는 확률 분포



3차원의 스위스 롤 데이터를 t-SNE를 이용하여 차원 축소

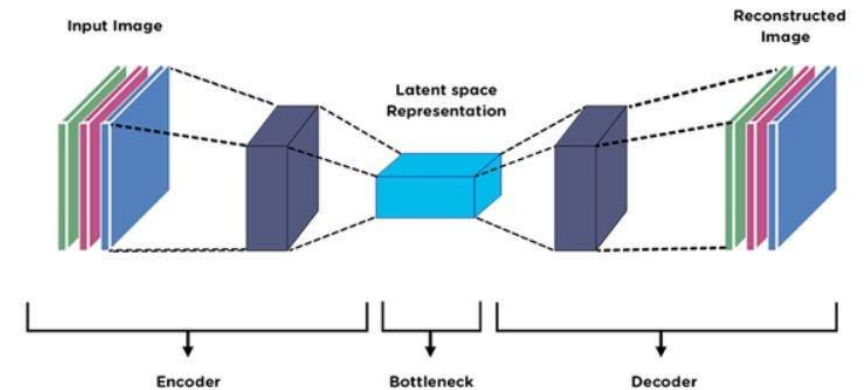
인코더·디코더 Encoder·Decoder

인코더

- 입력 데이터를 압축하여 Latent Space로 요약
- 입력을 신경망에 통과시키며, 중요한 패턴, 구조, 특징을 유지하면서 불필요한 정보를 제거

디코더

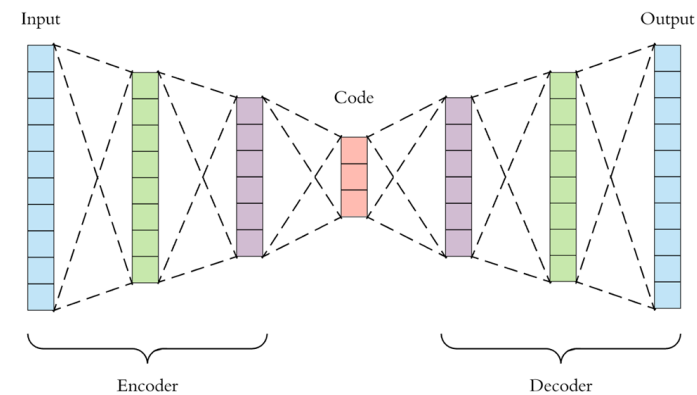
- Latent Space를 이용하여 원래의 데이터 복원
- 해당 특성을 이용하여 랜덤 노이즈에서 이미지 “복원” 가능
=> Stable Difusion 탄생



Encoder - Decoder
Architecture

오토인코더 Auto Encoder

- 입력 데이터의 패턴을 파악하여 최대한 유사한 데이터를 출력
- 입력된 값을 그대로 출력하는 것이 목적 => Self-Supervised Learning
- 구조 상, 새로운 데이터를 생성하는 능력이 제한적
=> VAE, GAN으로 발전



Auto Encoder의 구조

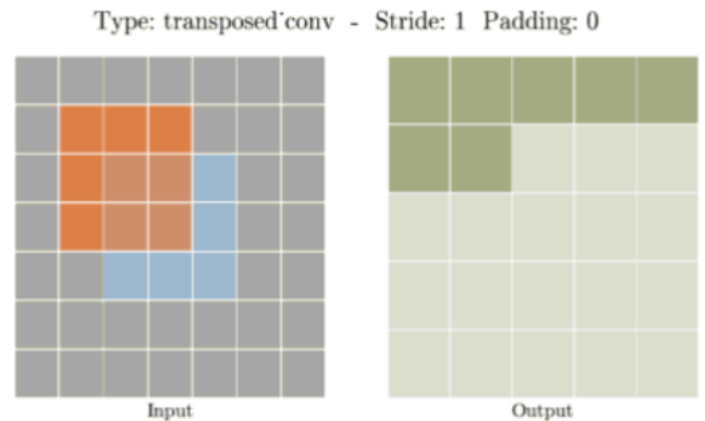
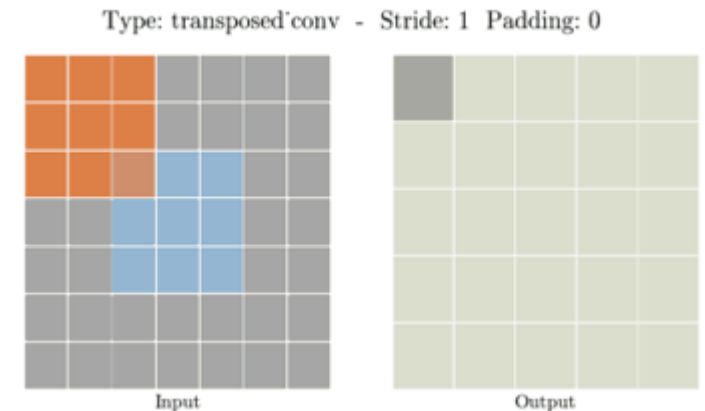
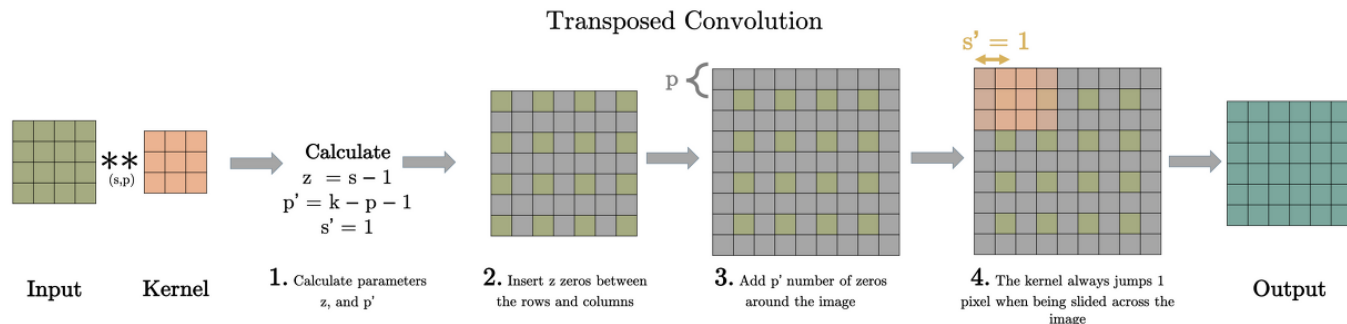
실습 #1) 앞선 실습에서 진행한 AE의 파라미터에 따른 결과 비교 분석

- Functional API를 사용하여 설계
- MNIST 데이터에 대해 AE 학습 진행 -> AE의 입력 크기를 (28x28)로 할 것
- 잠재 벡터의 차원 수에 따른 결과 비교 분석
- Encoder 및 Decoder의 계층 추가 및 구조 변경에 따른 결과 비교 분석
- 다양한 방법으로 계층 변경
ex) Convolutional layer 추가 등등

합성곱 오토인코더 Convolutional Auto Encoder

- 이미지 데이터에 대해, 위치에 대한 정보 또한 다루기 위해 컨볼루션 필터 이용
- 디코더에서 Transposed Convolution 이용

1. (stride - 1) 만큼 input 이미지의 열,행 사이에 zero padding을 실시해서 차원 증가
2. (kernel size - padding - 1)만큼 테두리에 zero padding을 실시해서 차원 증가
3. 차원을 증가시킨 input 이미지에 convolution 연산을 수행



Transposed Convolution의 진행

01 실습 #1 코드 분석

```
(x_train, _), (x_test, y_test) = keras.datasets.mnist.load_data()  
encoding_dim = 2
```

데이터를 준비한다.

y_test는 나중에 Latent Space를 시각화 할 예정인데, 그때 라벨링으로 쓰려고 준비했다.

```
x_train = x_train / 255  
x_test = x_test / 255  
x_train = x_train.reshape((len(x_train), 784))  
x_test = x_test.reshape((len(x_test), 784))
```

MNIST 데이터를 준비한다.

```
input_img = Input(shape=(784,), name='input_layer')  
encoded = Dense(encoding_dim, activation='relu', name='encoder_layer')(input_img)  
encoder = Model(inputs=input_img, outputs=encoded, name='encoder_model')  
encoded_input = Input(shape=(encoding_dim,), name='encoded_input')  
decoded = Dense(784, activation='relu', name='decoder_layer')(encoded_input)  
decoder = Model(inputs=encoded_input, outputs=decoded, name='decoder_model')  
encoded_output = encoder(input_img)  
decoded_output = decoder(encoded_output)  
AE = Model(inputs=input_img, outputs=decoded_output, name='autoencoder_model')
```

강의자료의 오토인코더 구조이다.

앞으로 이 오토인코더 구조를 바닐라 오토인코더라고 부르도록 하겠다.

keras의 Sequential 대신 Functional API를 이용하였다.

01 실습 #1 코드 분석

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))

# CNN 추가 버전 오토인코더
input_img = Input(shape=(28, 28, 1))

x = Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = MaxPooling2D((2, 2), padding='same')(x)
x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = MaxPooling2D((2, 2), padding='same')(x)
encoder = Model(inputs=input_img, outputs=x, name='encoder_model')

x = Conv2DTranspose(32, (3, 3), strides=2, activation="relu", padding="same")(x)
x = Conv2DTranspose(32, (3, 3), strides=2, activation="relu", padding="same")(x)
x = Conv2D(1, (3, 3), activation="sigmoid", padding="same")(x)
decoded = Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

AE = Model(input_img, decoded)
```

컨볼루션 레이어를 추가한 오토인코더이다.

앞으로 이 오토인코더를 CNN 오토인코더라고 부르도록 하겠다.

MaxPooling같은 경우는 역방향으로 되돌아갈 수 없는 작업이므로, 디코더에서는 Transposed Convolution을 사용하였다.

앞서 바닐라 오토인코더와 마찬가지로 입력값을 정규화 해주었는데, 마찬가지로 각 픽셀의 값이 0~1 사이에 있을 것이며,

따라서 CNN 오토인코더의 출력층은 시그모이드를 사용한 것을 알 수 있다.

01 실습 #1 코드 분석

```
AE.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
early_stopping = keras.callbacks.EarlyStopping(monitor='loss', patience=20)
history = AE.fit(x_train, x_train, epochs=50, batch_size=256, shuffle=True, callbacks=early_stopping)
decoded_img = AE.predict(x_test)
```

데이터를 준비할 때 255로 나눠 0~1 사이의 값으로 정규화를 하였다.
각 픽셀은 0 또는 1에 가까운 값을 가지므로, MSE 대신 CEE를 사용하였다.
옵티마이저는 몇 번 실행 결과 adam의 성능이 제일 괜찮았다.
역시 Early Stopping을 세팅했고, patience 값은 20으로 세팅했다.

```
from tensorflow.keras.utils import plot_model
plot_model(AE, to_file=save_path, show_shapes=True, show_layer_names=True)
```

모델의 구조를 이미지로 저장하는 코드이다.

```
n = 4
plt.figure(figsize=(10, 6))
plt.suptitle(f'Latent Vector: {encoding_dim}\nepoch: 50, batch size: 256')
for i in range(1, n+1):
    ax = plt.subplot(2, n, i)
    plt.title("Original")
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')

    ax = plt.subplot(2, n, i+n)

    plt.title("Predict")
    plt.imshow(decoded_img[i].reshape(28, 28), cmap='gray')
```

원본 이미지와 예측된 이미지를 표시하는 코드이다.
epoch과 batch size는 변경하지 않았으며, encoding_dim값은 2와 256 두 가지 값을 가진다.

01 실습 #1 코드 분석

```
from sklearn.manifold import TSNE
from sklearn.decomposition import PCA
latent_representations = encoder.predict(x_test)
latent_resaped = latent_representations.reshape(len(latent_representations), -1)
# t-sne
# tsne = TSNE(n_components=2, random_state=42)
# latent_2d = tsne.fit_transform(latent_representations)
# pca
pca = PCA(n_components=2)
latent_2d = pca.fit_transform(latent_resaped)
plt.figure(figsize=(10, 8))
plt.scatter(latent_2d[:, 0], latent_2d[:, 1], c=y_test, cmap='gist_rainbow', s=5, alpha=0.5)
plt.colorbar()
plt.title('Latent Space Visualization')

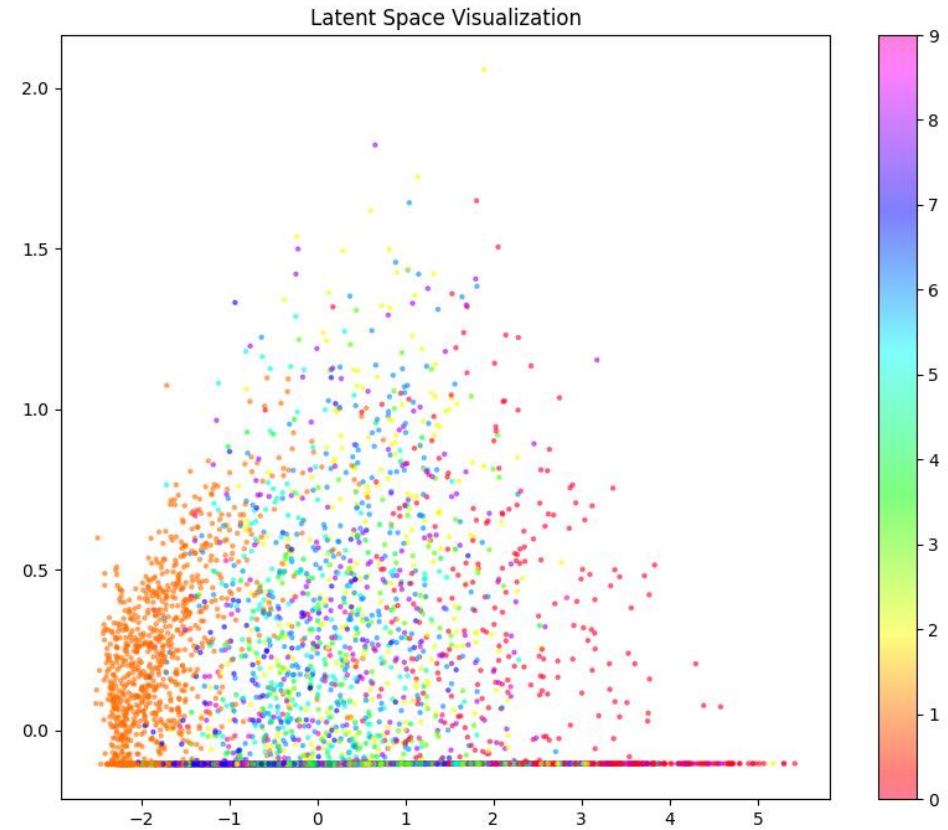
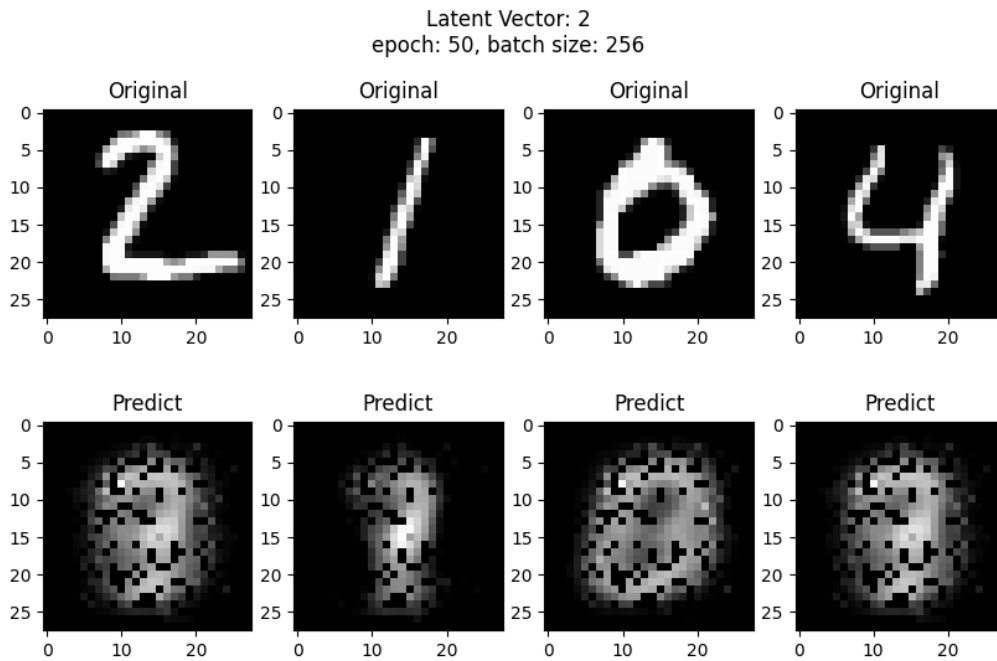
plt.show()
```

Latent Space를 시각화하는 코드이다.

2차원 공간에 Latent Space를 나타내었으며, 이를 위해 차원 축소 방법으로는 t-SNE와 PCA를 선택하였다.

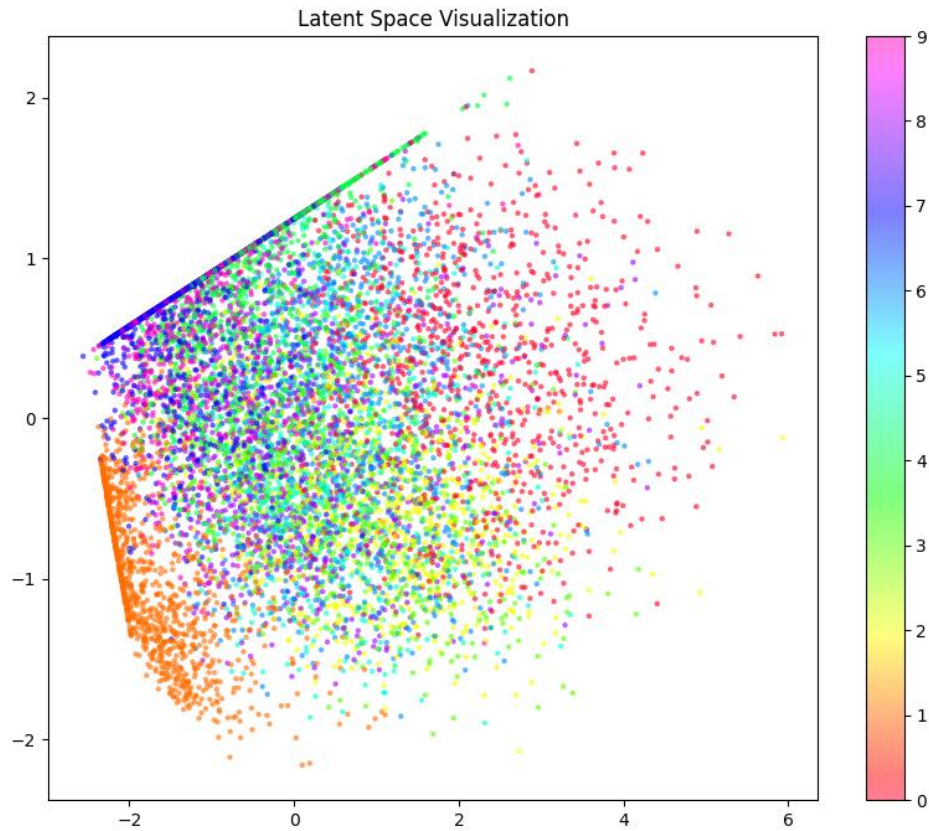
다만 데이터의 개수가 많은 관계로 주로 PCA로 실행하였다.

Vanilla Autoencoder

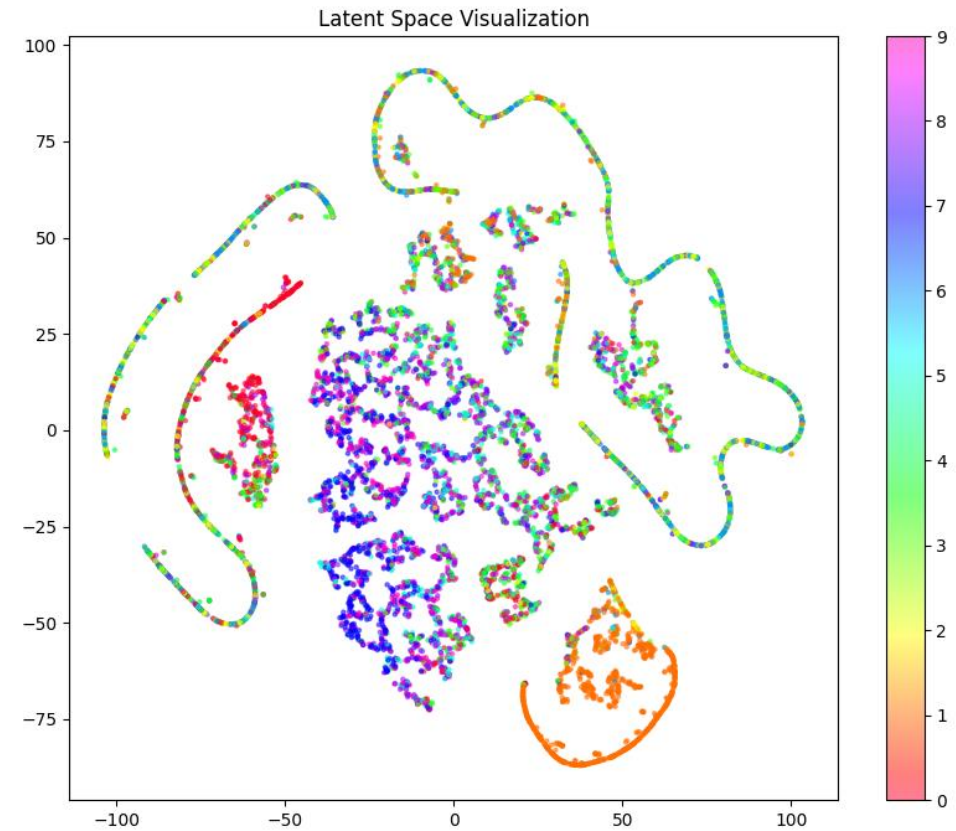


데이터들이 한 축 위에 몰린걸 볼 수 있는데,
PCA 과정에서 차원 축소가
제대로 이루어지지 않았다고 예상된다.

Vanilla Autoencoder

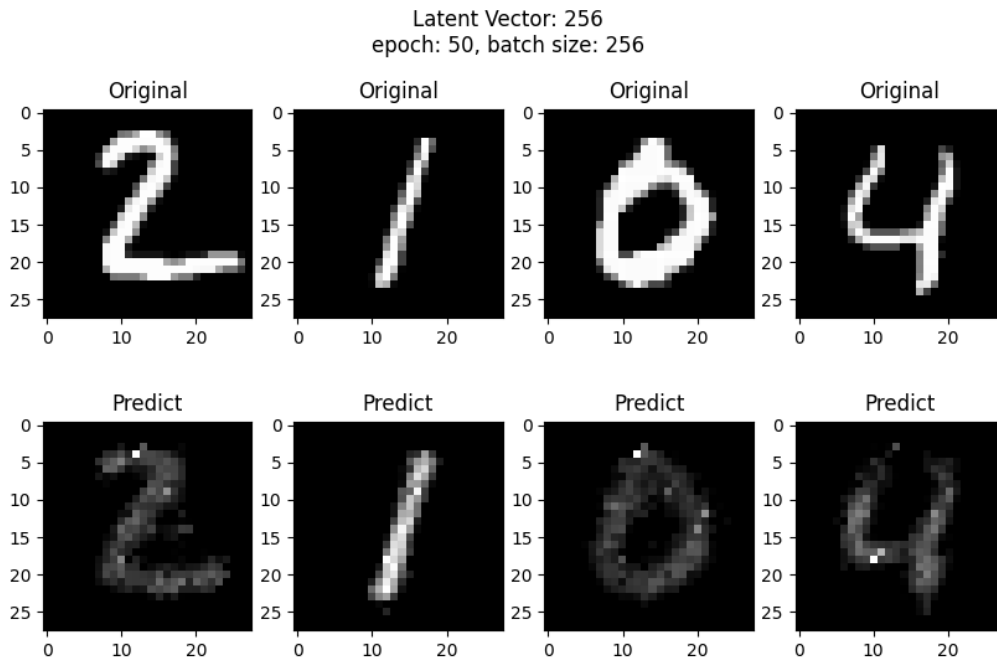


PCA를 사용했을 때

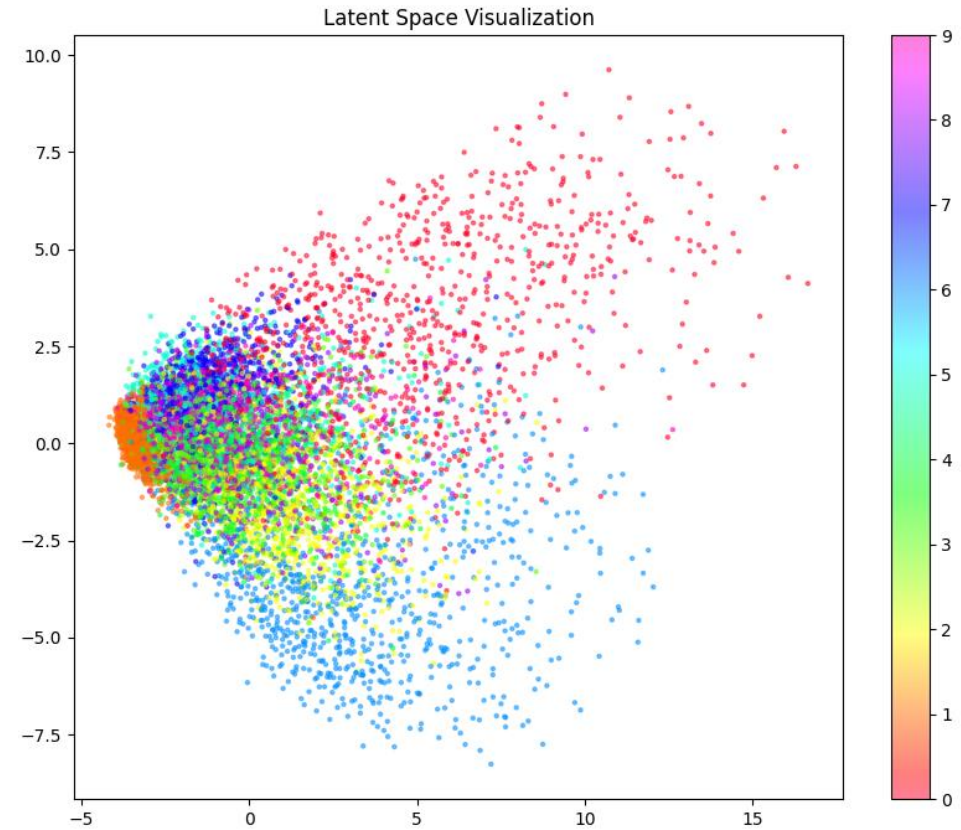


t-SNE를 사용했을 때

Vanilla Autoencoder

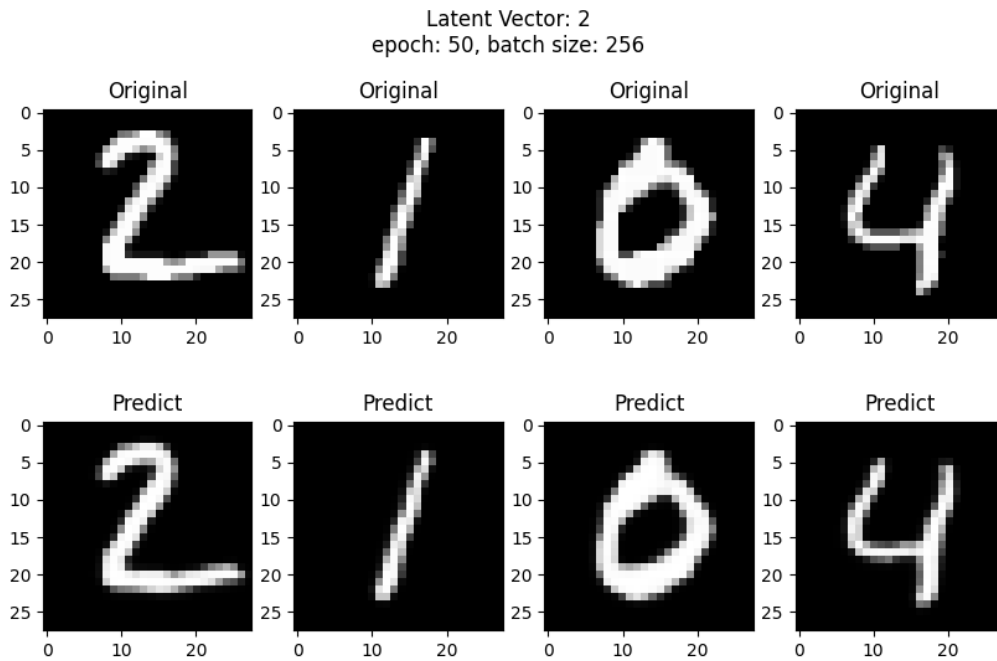


encoding_dim이 256인 이유는 별 건 없고,
그냥 숫자가 마음에 들었기 때문이다.

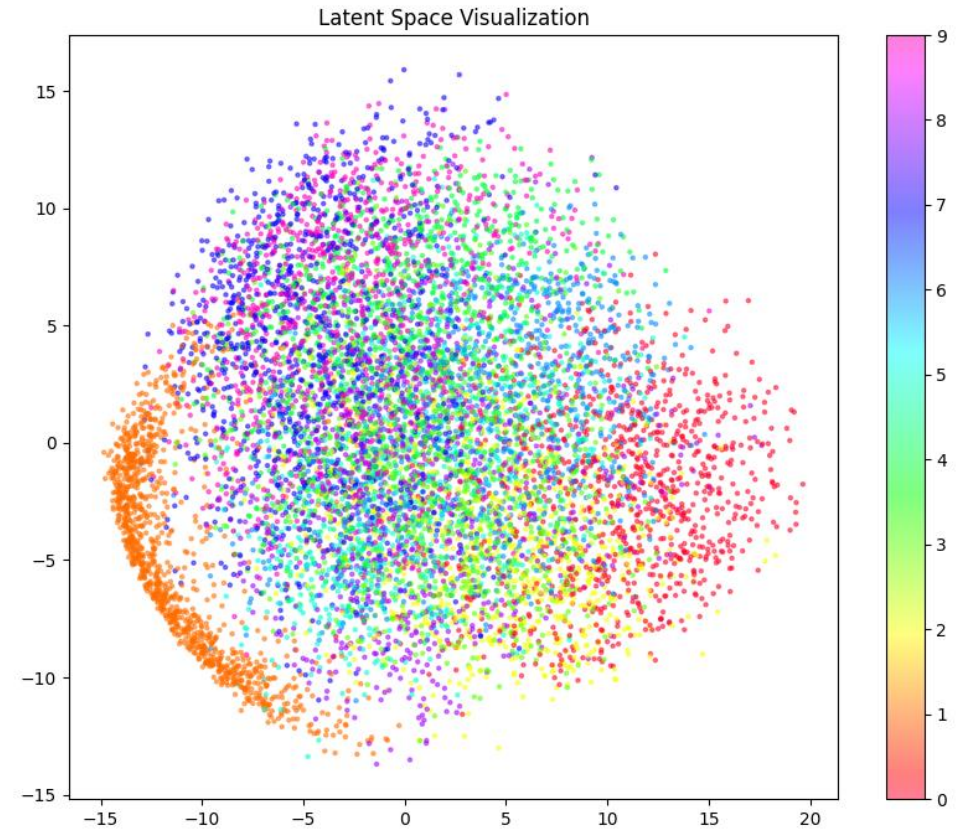


색들이 좀 모인거같긴 한데
그래도 뭔가 부족한 모습이다.

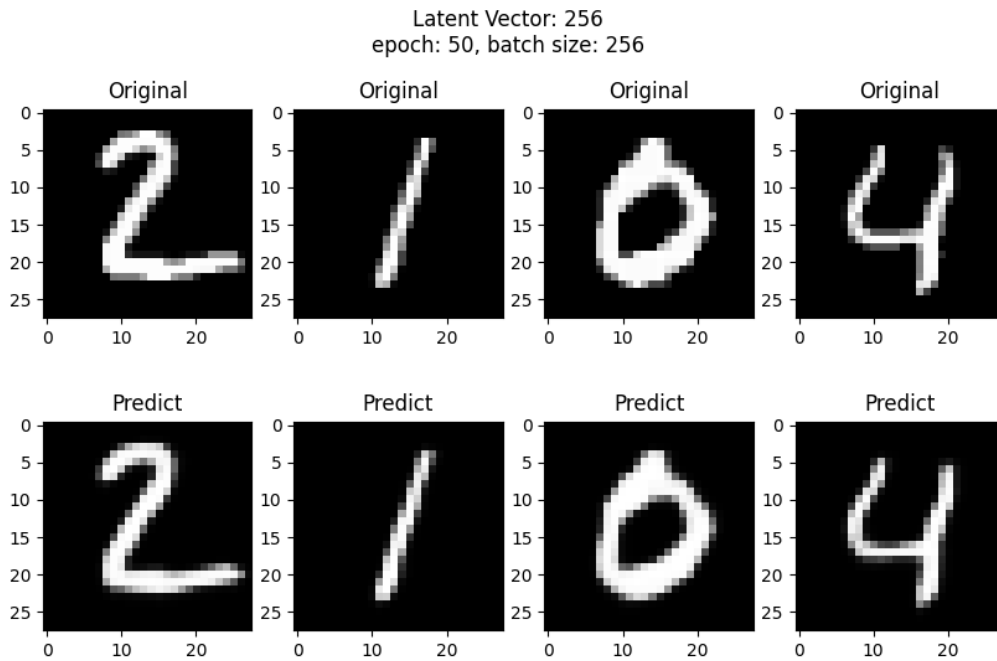
CNN Autoencoder



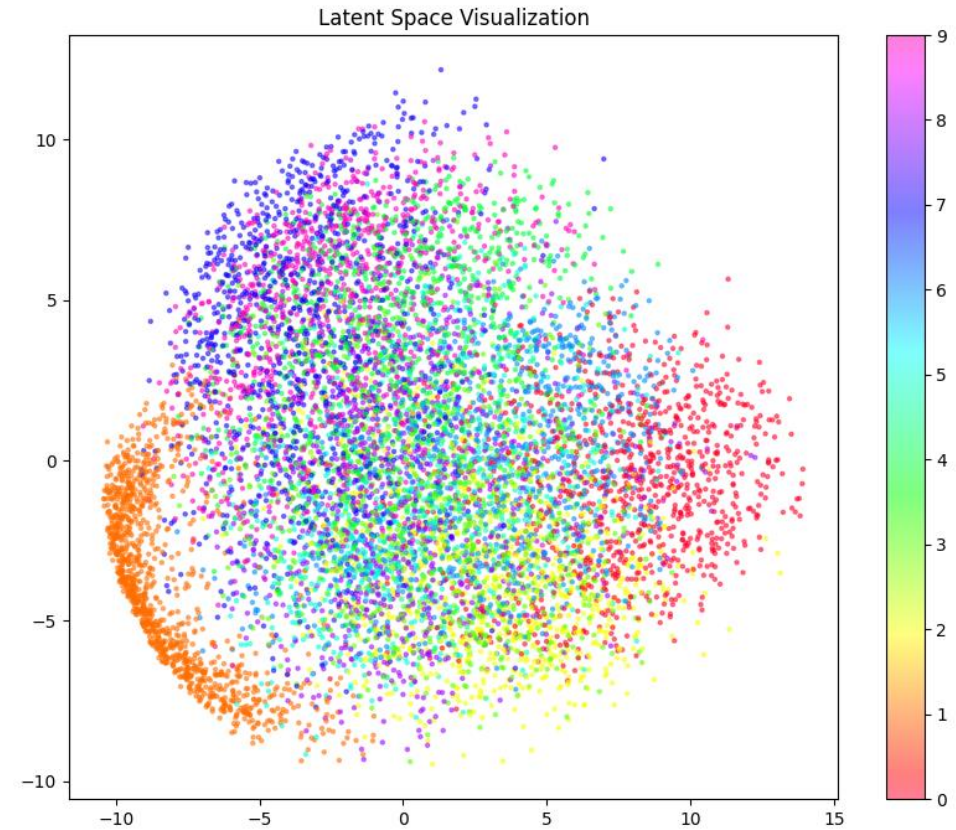
아까 바닐라 오토인코더보다 훨씬 깔끔한 출력이다.
컨볼루션 레이어가 들어갔기 때문에 공간에 대한
정보도 얻을 수 있기 때문이라고 추정된다.



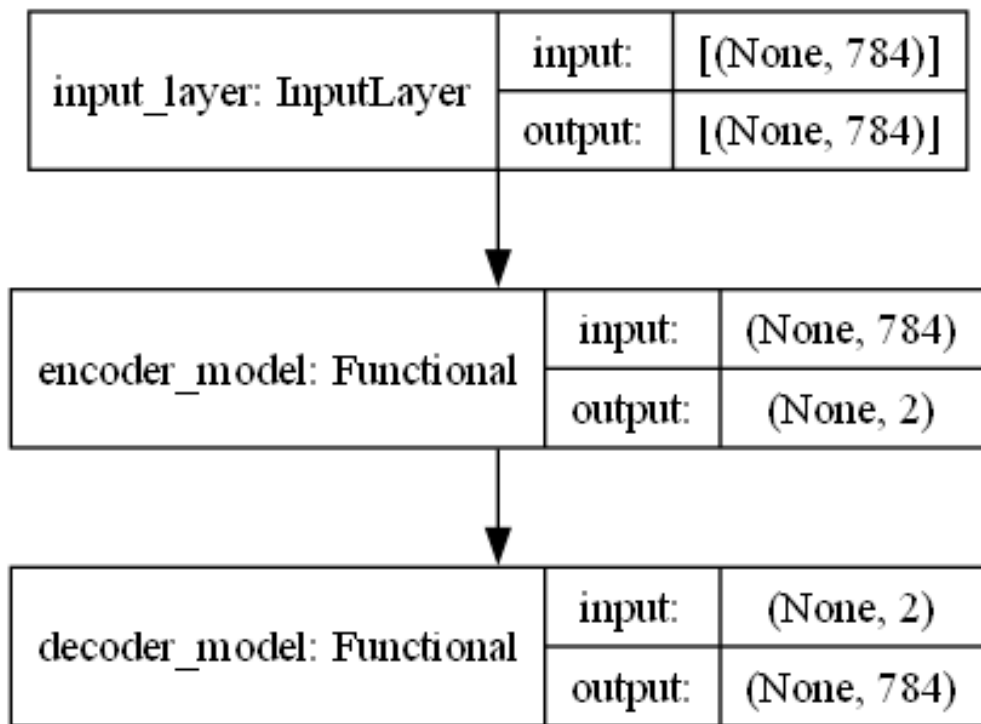
CNN Autoencoder



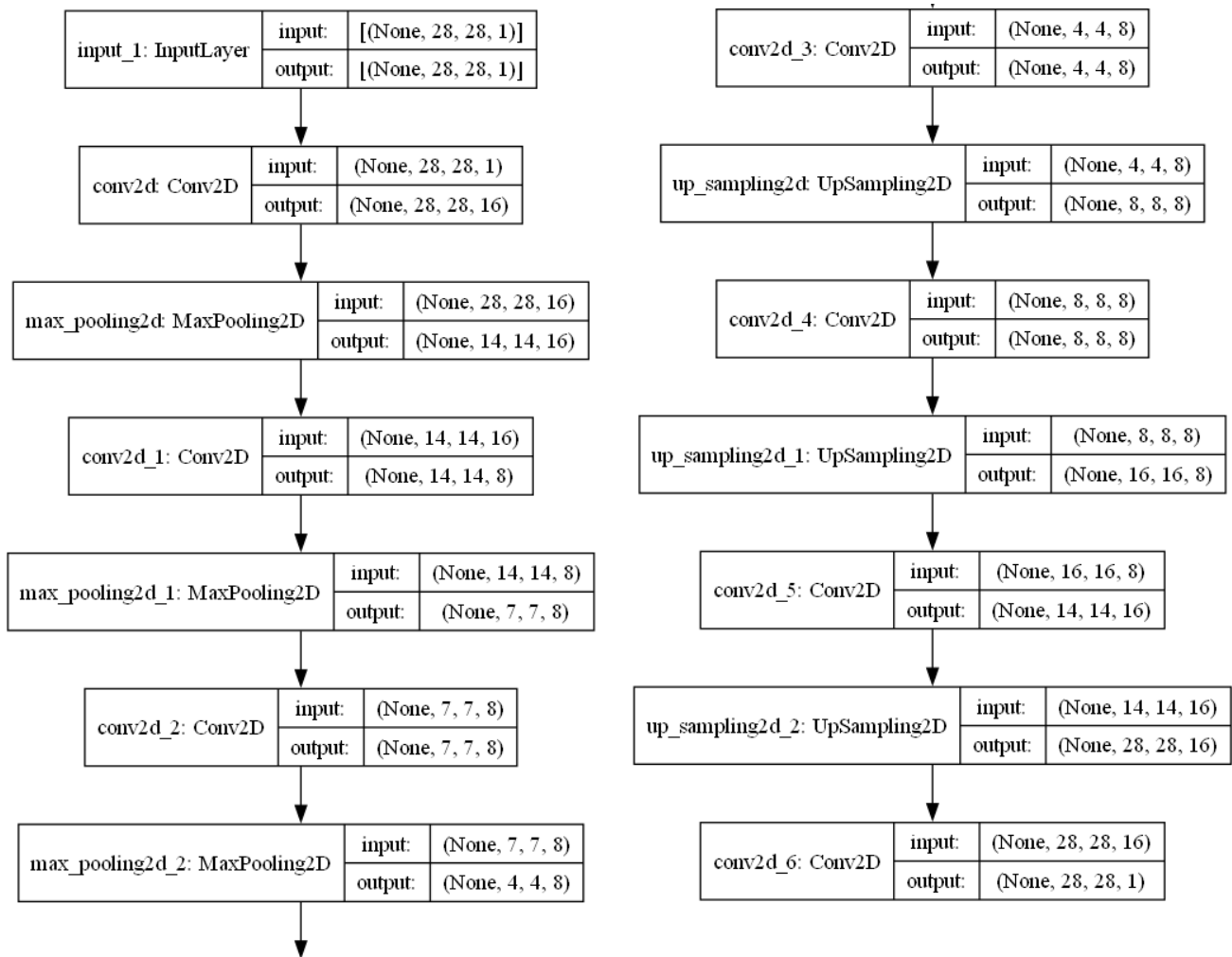
바닐라 오토인코더와 CNN 오토인코더 둘 다
encoding_dim이 증가했을 때 예측이 더 정확했다.



Autoencoder Architecture



Vanilla Autoencoder



CNN Autoencoder

실습 #2) 노이즈를 제거하는 AE 만들기

- Functional API를 사용하여 설계
- MNIST 데이터에 노이즈 추가
- 노이즈 데이터를 입력, 원본 데이터를 Target으로 진행
- 입력 이미지, 원본 이미지, 복원 이미지를 비교

01 실습 #2 코드 분석

```
x_train = x_train / 255
x_test = x_test / 255
x_train = x_train.reshape((len(x_train), 784))
x_test = x_test.reshape((len(x_test), 784))
noisy_train = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
noise_train = np.clip(noisy_train, 0., 1.)
noisy_test = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)
noise_test = np.clip(noisy_test, 0., 1.)
```

나머지는 아까와 동일한데, 데이터를 만드는 부분이 약간 달라졌다.
입력에 노이즈를 추가하는 코드를 추가했으며, noise_factor 의 값은 0.9로 설정했다.

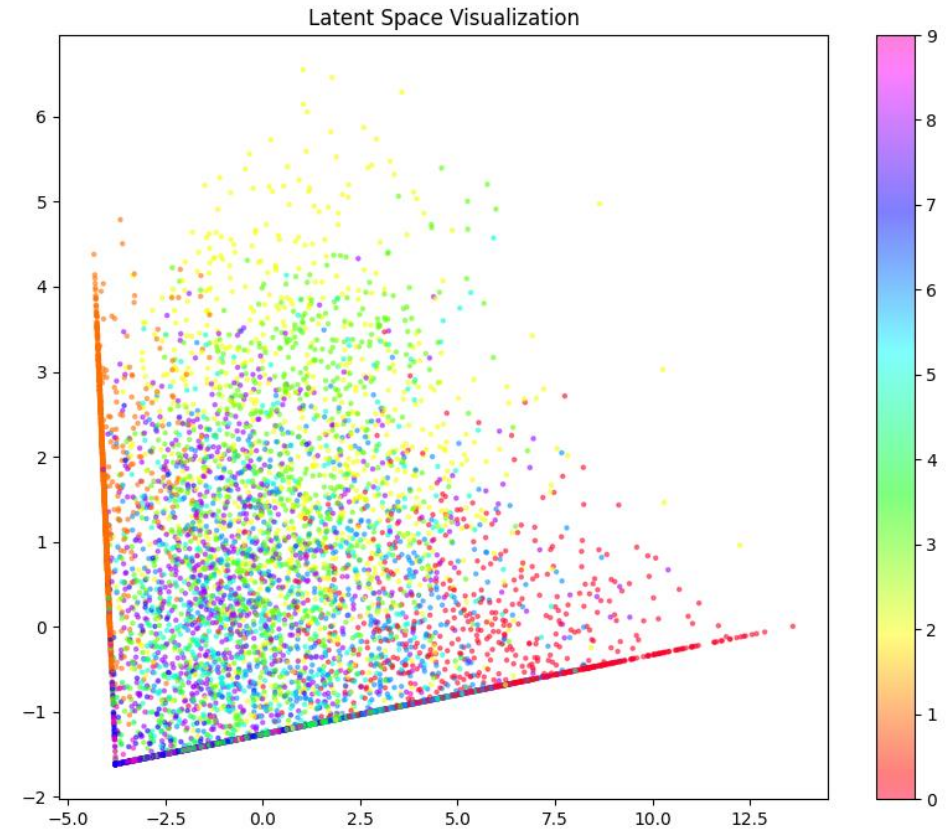
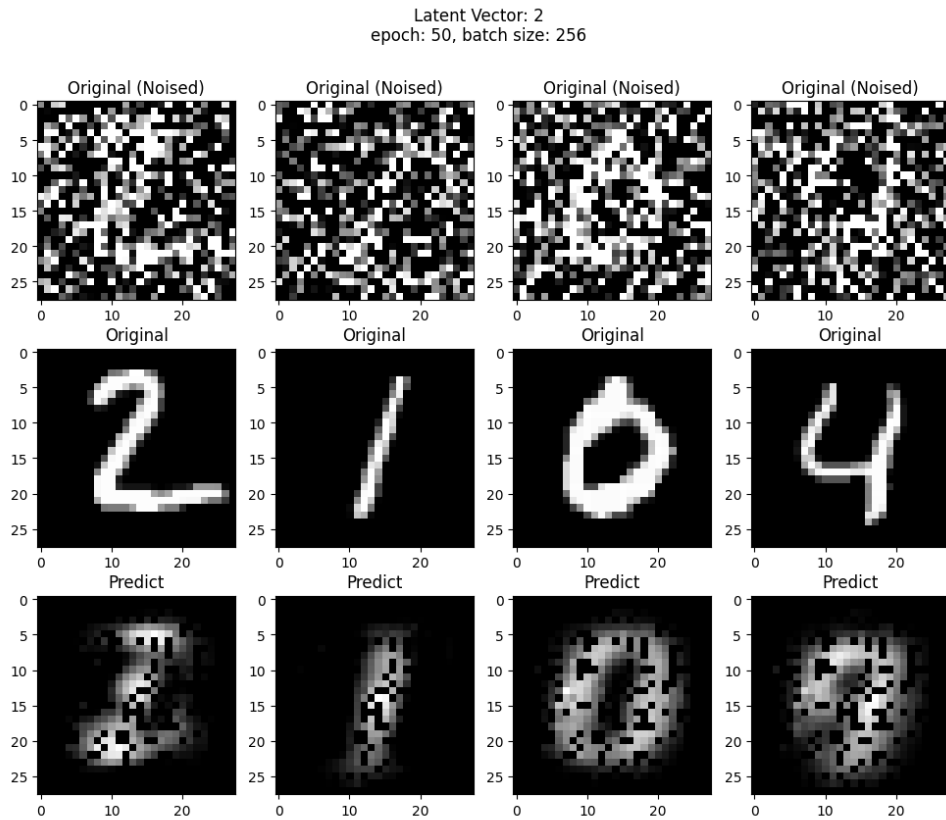
```
n = 4
plt.figure(figsize=(12, 9))
plt.suptitle(f'Latent Vector: {encoding_dim}\nepoch: 50, batch size: 256')
for i in range(1, n+1):
    ax = plt.subplot(3, n, i)
    plt.title("Original (Noised)")
    plt.imshow(noise_test[i].reshape(28, 28), cmap='gray')

    ax = plt.subplot(3, n, i+n)
    plt.title("Original")
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')

    ax = plt.subplot(3, n, i+n+n)
    plt.title("Predict")
    plt.imshow(decoded_img[i].reshape(28, 28), cmap='gray')
```

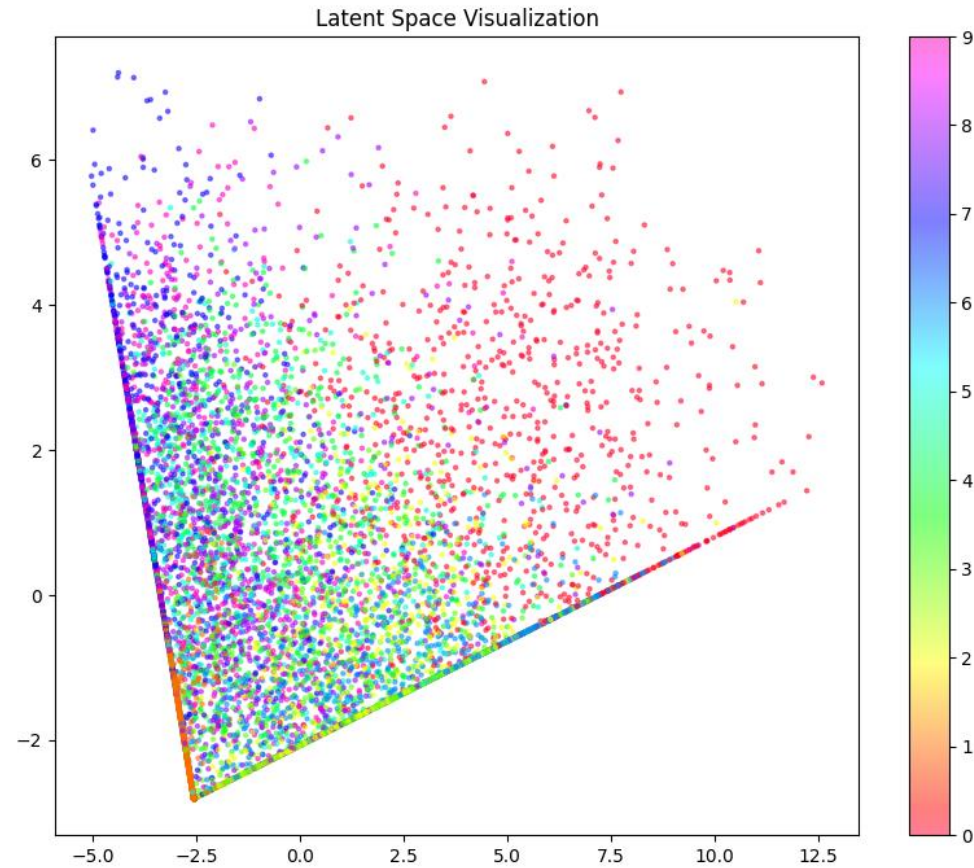
노이즈가 추가되었기 때문에 이쪽의 코드도 수정했다.
이제 기존의 2행 4열이 아닌 3행 4열로 표시된다.

Vanilla Autoencoder



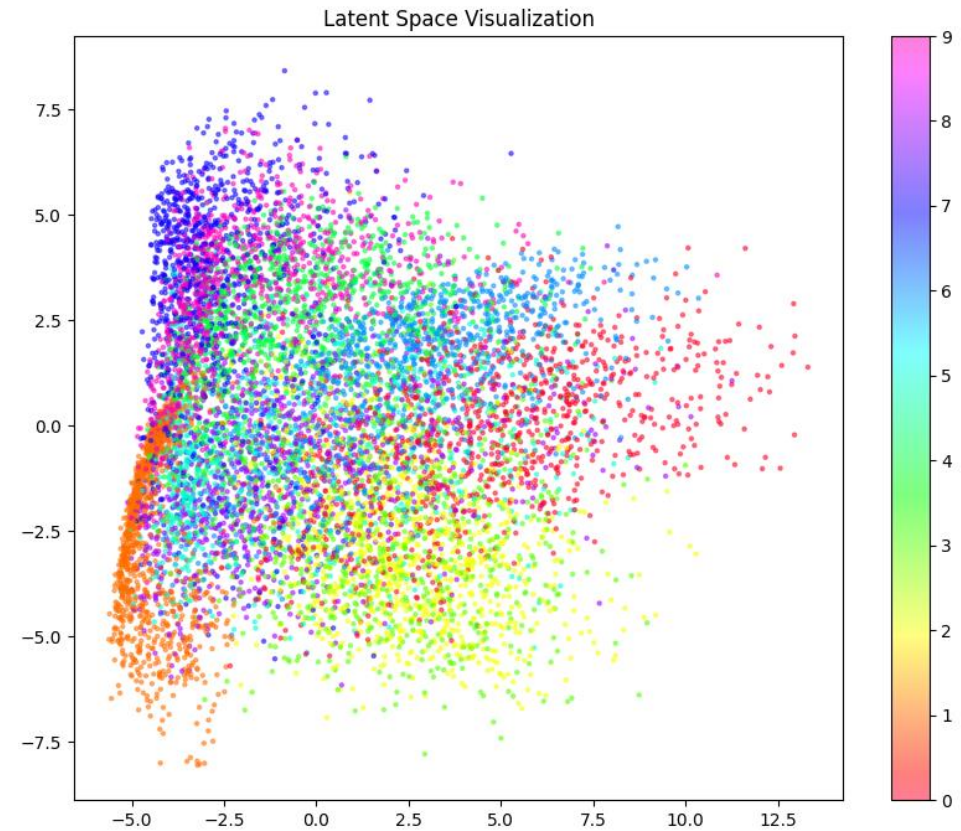
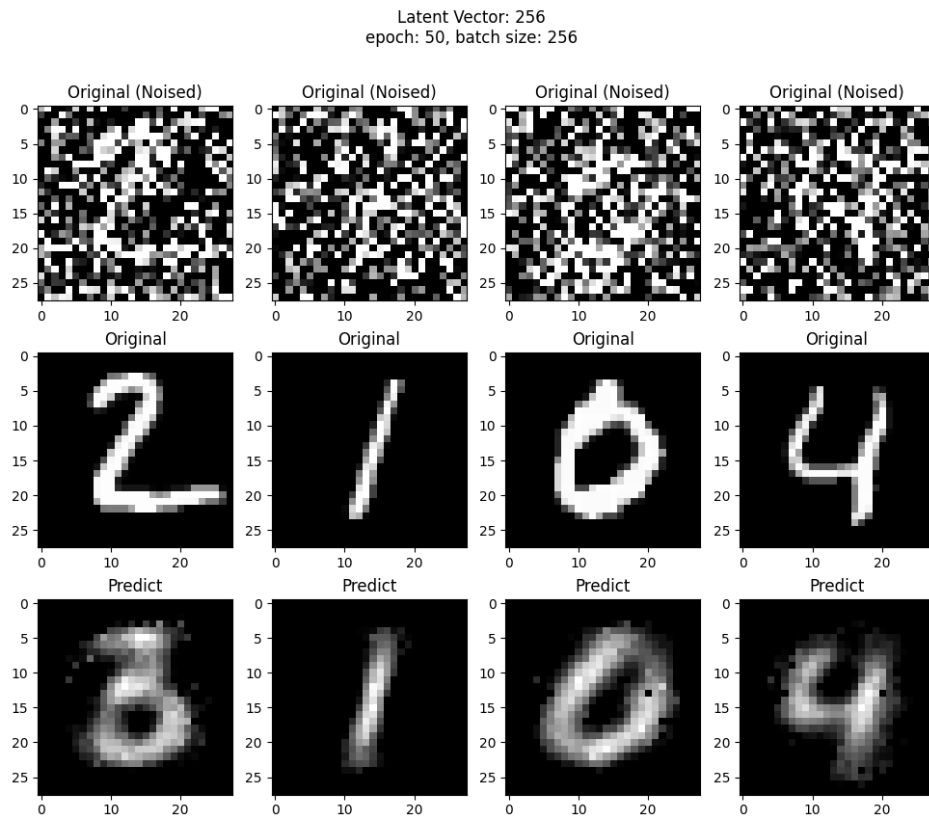
PCA로 차원축소한건데,
여전히 주성분을 잘 찾지 못하여
한 축에 데이터가 몰린걸 볼 수 있다.

Vanilla Autoencoder



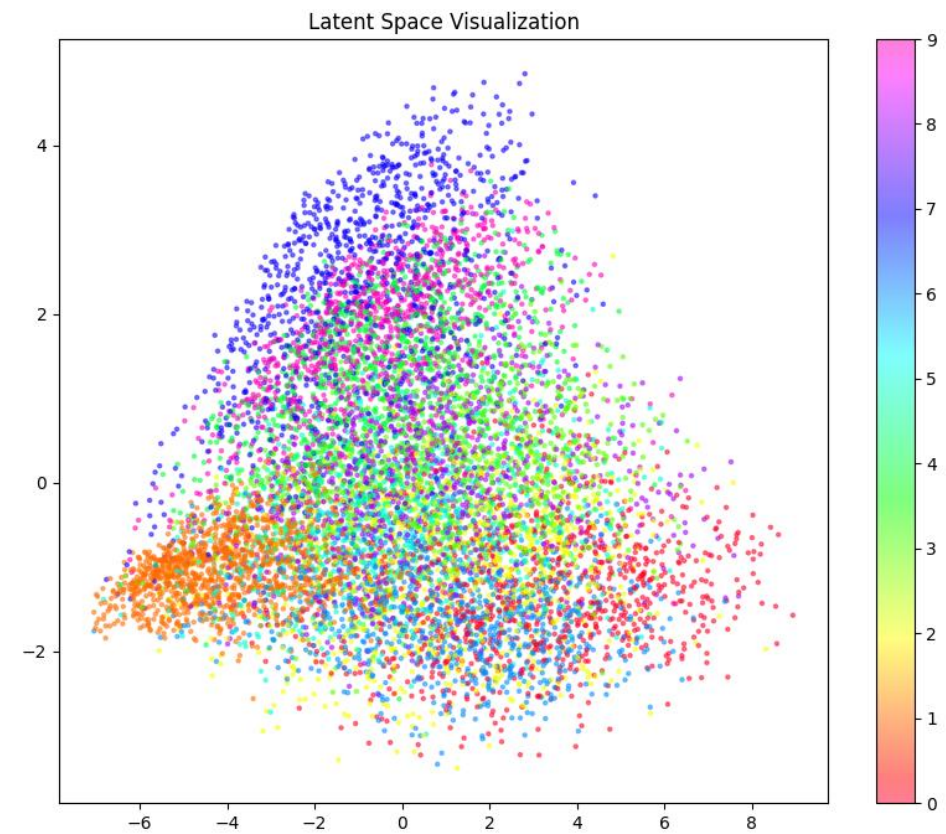
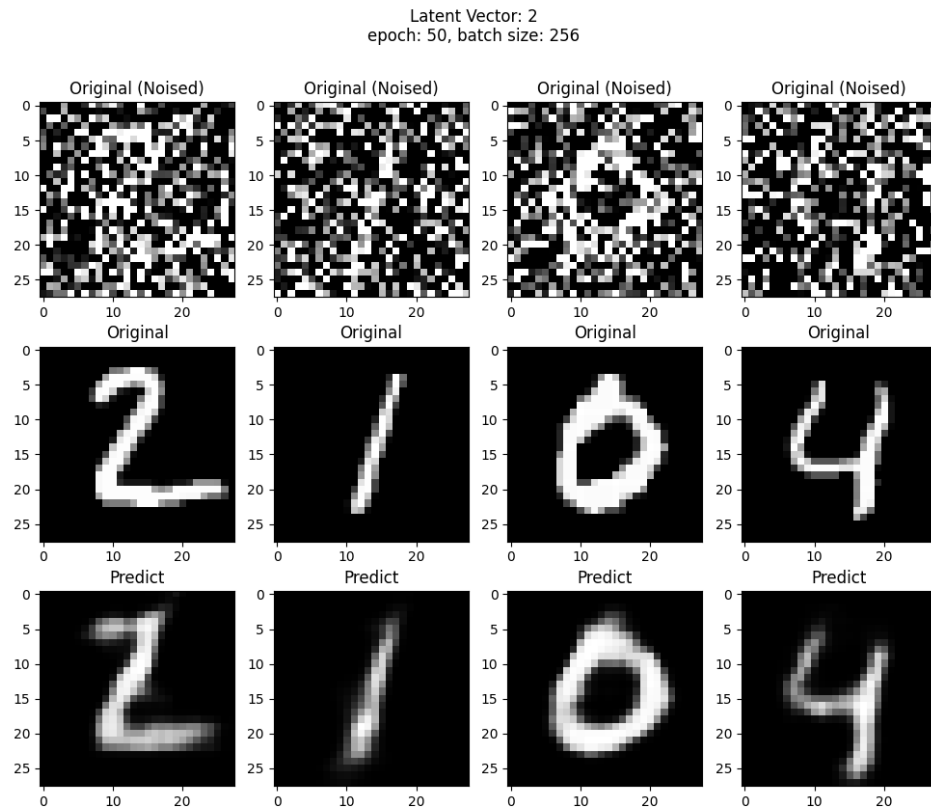
이거는 t-SNE인데, 이 방법도 마찬가지로
한 축에 데이터가 몰린 걸 볼 수 있다.

Vanilla Autoencoder

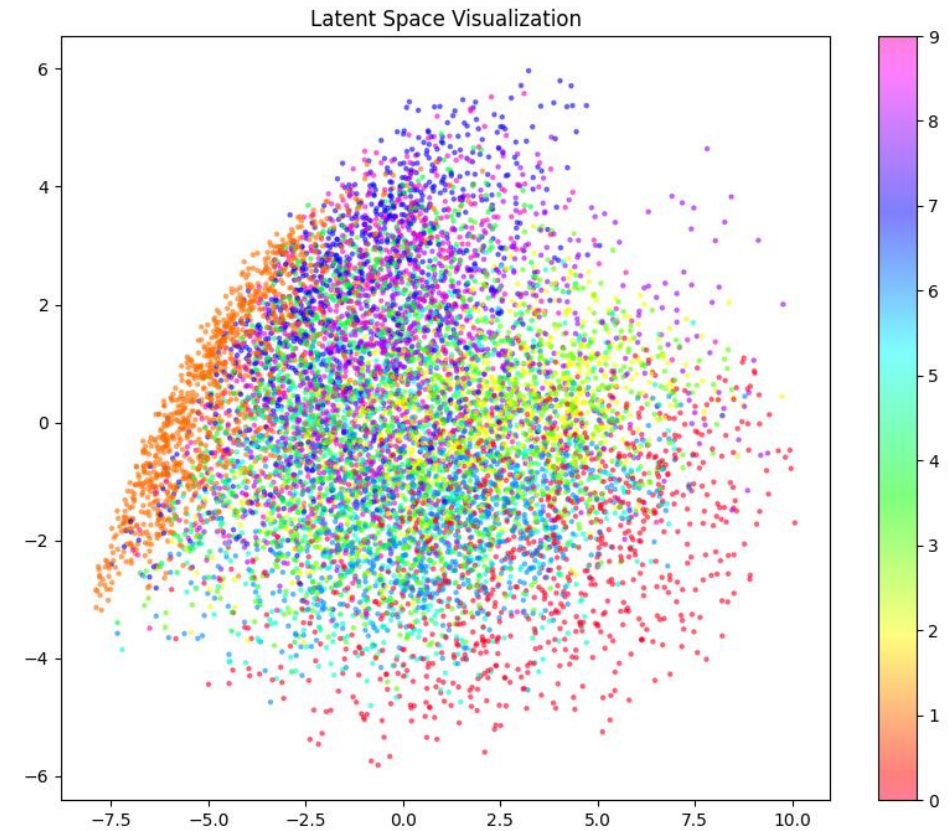
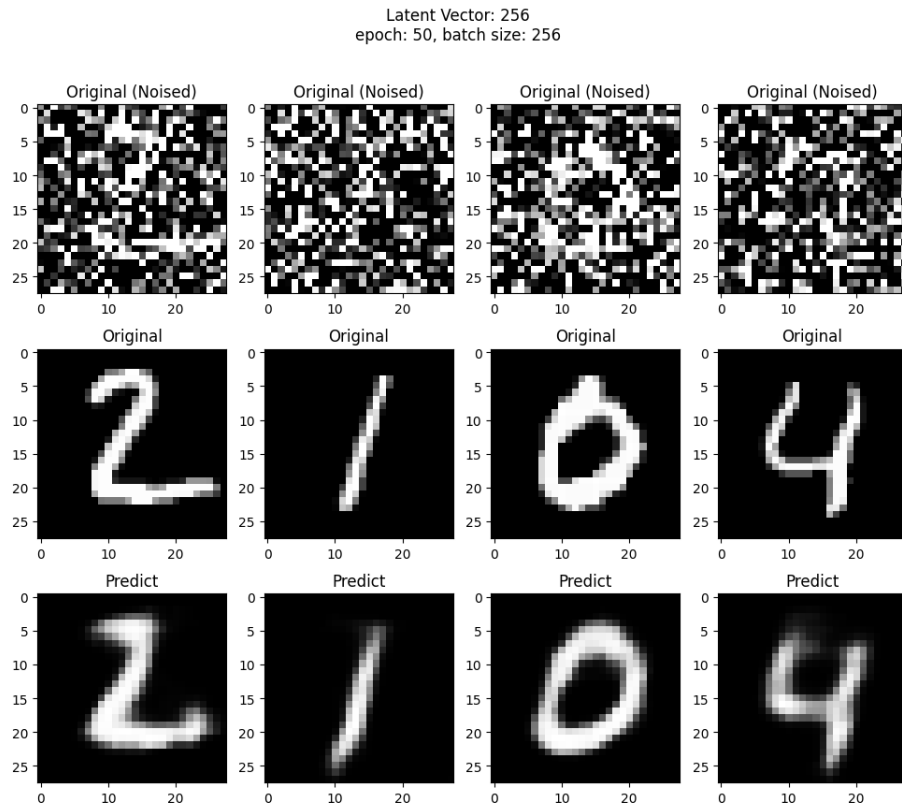


encoding_dim을 좀 늘리니까
확실히 아까보다는 나아진 모습이다.

CNN Autoencoder



CNN Autoencoder



CNN은 이미지에 최적화되어서 그런지
노이즈가 이리 심해도 엄청난 성능을 내는걸 알 수 있다.

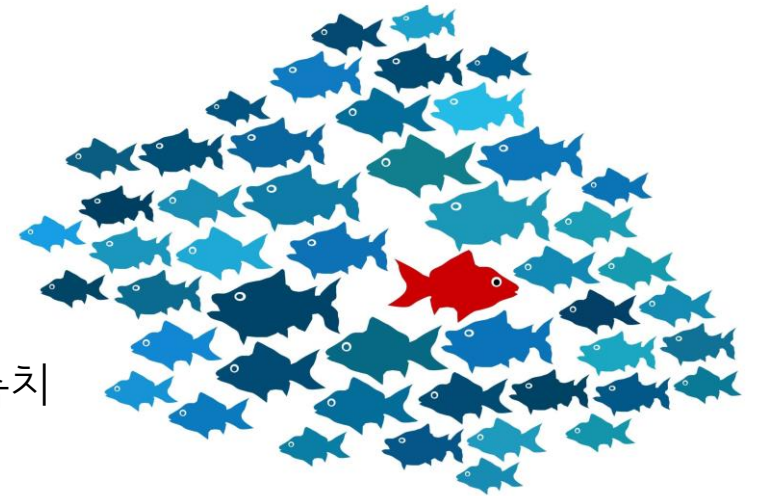
02

이상치 탐지

Anomaly Detection

이상치 Outlier

- 데이터 집합에서 일반적인 패턴에서 크게 벗어나는 관측값
- Global Outlier: 전체 데이터 집합에서 다른 데이터와 현저하게 다른 값을 가진 관측치
- Contextual Outlier: 특정 맥락이나 조건에서만 이상으로 간주되는 관측치



이상치의 예시

사하라 사막에서의 40도



Global: 정상
Contextual: 정상

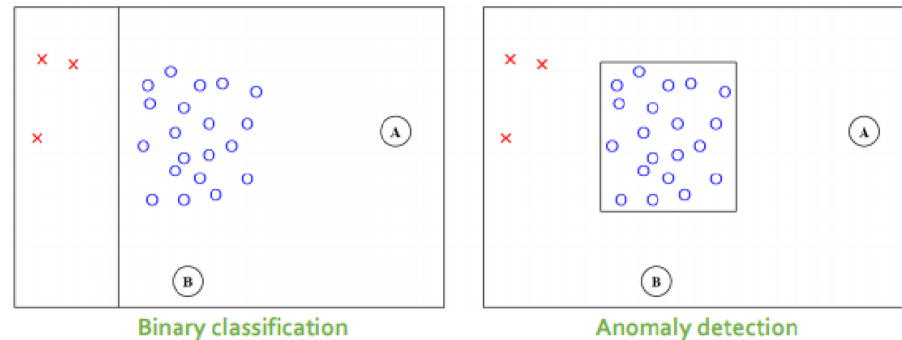
알래스카에서의 40도



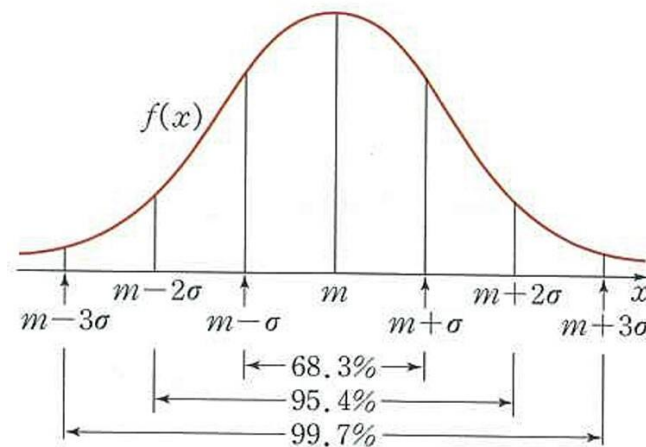
Global: 정상 (지구의 어딘가는 40도이므로)
Contextual: **Outlier**

이상치 탐지 Anomaly/Novelty Detection

- 데이터 집합에서 정상 패턴과 현저하게 다른 데이터를 식별하는 과정
- Z-Score, Machine Learning, Auto Encoder 등을 이용하여 탐지
- Supervised Learning: 정상 데이터와 이상치에 대한 레이블이 있는 경우. 분류 알고리즘을 사용하여 이상치를 탐지
- Unsupervised Learning: 레이블이 없는 경우. 군집화 알고리즘이나 밀도 기반 방법을 통해 이상치를 식별
- Semi - Supervised Learning: 일부 데이터에만 레이블이 있는 경우. 정상 데이터로 모델을 학습시키고, 새로운 데이터가 정상 패턴에서 벗어나는지를 평가하여 이상치를 탐지



Classification과
Anomaly Detection

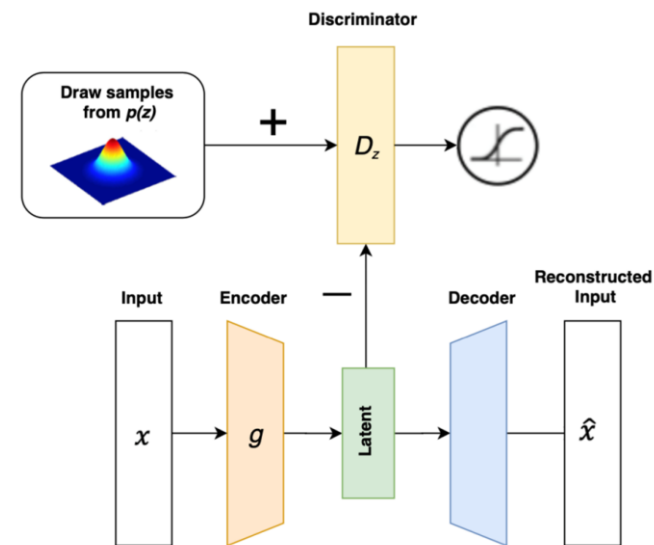


표준정규분포표와 z값

AE 이상치 탐지

Anomaly Detection with Auto Encoder

1. 오토인코더를 정상 데이터로만 학습시켜 입력 데이터를 효과적으로 재구성
2. 새로운 데이터가 입력되면, 오토인코더는 이를 기반으로 출력을 생성하고, 입력과 출력 간의 재구성 오차를 계산
3. 오차가 사전에 설정한 Threshold를 초과하면 해당 데이터를 이상치로 간주
이는 오토인코더가 정상 데이터의 패턴을 학습했기 때문에,
비정상 데이터는 제대로 재구성하지 못해 오차가 크게 발생하기 때문



Auto Encoder를 이용한 이상치 탐지

실습 #3) AE를 활용하여 Anomaly Detection을 구현

- 인공적으로 만들어진 시계열 데이터 활용
- 아래 Keras Link를 참고하여 시계열 데이터의 Anomaly Detection 구현
- 설명을 잘 숙지하고 여러 구조 및 파라미터 조절에 따른 결과 비교 분석 ex) threshold 등
- 단순 과제 제출 목적이 아닌, Anomaly Detection에 대한 이해를 목적으로 진행 요망

02 실습 #3 코드 분석

```
import numpy as np
import pandas as pd
import keras
from keras import layers
from matplotlib import pyplot as plt

master_url_root = "https://raw.githubusercontent.com/numenta/NAB/master/data/"

# 이상치 없는 데이터
df_small_noise_url_suffix = "artificialNoAnomaly/art_daily_small_noise.csv"
df_small_noise_url = master_url_root + df_small_noise_url_suffix
df_small_noise = pd.read_csv(df_small_noise_url, parse_dates=True, index_col="timestamp")
# 이상치 있는 데이터
df_daily_jumpsup_url_suffix = "artificialWithAnomaly/art_daily_jumpsup.csv"
df_daily_jumpsup_url = master_url_root + df_daily_jumpsup_url_suffix
df_daily_jumpsup = pd.read_csv(df_daily_jumpsup_url, parse_dates=True, index_col="timestamp")

training_mean = df_small_noise.mean()
training_std = df_small_noise.std()
df_training_value = (df_small_noise - training_mean) / training_std
TIME_STEPS = 288
```

해당 코드는 [여기](#)에서 가져온 오토인코더로 시계열 데이터의 이상치를 탐지하는 코드이다.

코드의 링크에서 시계열 데이터를 가져온다.

df_small_noise는 이상치가 없는 데이터이고, df_daily_jumpsup은 이상치가 포함된 데이터이다.

이상치가 없는 데이터의 평균과 표준편차를 구하고, 이를 이용하여 평균을 0, 표준편차를 1인 형태로 가공한다.

이는 학습 동안 데이터의 크기를 균일하게 하기 위함이다.

TIME_STEPS가 288이므로, 288 time step의 시퀀스를 생성한다.

02 실습 #3 코드 분석

```
def create_sequences(values, time_steps=TIME_STEPS):  
    output = []  
    for i in range(len(values) - time_steps + 1):  
        output.append(values[i : (i + time_steps)])  
    return np.stack(output)
```

해당 함수로 시퀀스를 생성한다.

아까 정의한 TIME_STEPS를 불러와 288개의 연속적 데이터 포인트로 시퀀스를 구성한다.

```
x_train = create_sequences(df_training_value.values)  
model = keras.Sequential([  
    # Encoder  
    layers.Input(shape=(x_train.shape[1], x_train.shape[2])),  
    layers.Conv1D(filters=32, kernel_size=7, padding="same", strides=2, activation="relu",),  
    layers.Dropout(rate=0.2),  
    layers.Conv1D(filters=16, kernel_size=7, padding="same", strides=2, activation="relu",),  
    # Decoder  
    layers.Conv1DTranspose(filters=16, kernel_size=7, padding="same", strides=2, activation="relu",),  
    layers.Dropout(rate=0.2),  
    layers.Conv1DTranspose(filters=32, kernel_size=7, padding="same", strides=2, activation="relu",),  
    layers.Conv1DTranspose(filters=1, kernel_size=7, padding="same"),  
])
```

이상치 탐지를 위한 오토인코더 모델을 정의하는 부분이다.

입력 데이터의 형태를 정의하고, 커널 크기 7, 필터 개수 32를 이용하여 데이터를 압축한다.

이후, 20% 뉴런을 무작위로 비활성화하고 컨볼루션 레이어로 한번 더 압축하면 인코더가 완성된다.

디코더는 Convolution 대신 Transposed Convolution을 이용하여 데이터의 차원을 다시 복원한다.

02 실습 #3 코드 분석

```
model.compile(optimizer='adam', loss="mse")
history = model.fit(x_train, x_train, epochs=50, batch_size=128, validation_split=0.1,
callbacks=[keras.callbacks.EarlyStopping(monitor="val_loss", patience=5, mode="min")],)
```

모델은 adam 옵티마이저, mse 오차를 사용한다.

이후 모델을 학습하는데, Early Stopping을 이용하여 조기에 학습이 종료되도록 한다.

```
from tensorflow.keras.utils import plot_model
plot_model(model, to_file=save_path, show_shapes=True, show_layer_names=True)
```

모델의 구조를 이미지로 저장한다.

```
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Training and Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid()
plt.show()
```

학습 중 나온 Training Loss와 Validation Loss를 그래프로 plot 한다.

02 실습 #3 코드 분석

```
x_train_pred = model.predict(x_train)
train_mae_loss = np.mean(np.abs(x_train_pred - x_train), axis=1)
```

학습된 모델을 사용해 훈련 데이터를 재구성하고, 재구성된 데이터와 원본 데이터의 차이를 구해 MAE (MSE에서 제곱이 절대값으로 바뀐)를 계산한다. MSE 대신 MAE를 쓰는 이유는 이상치는 너무 일반적이지 않은 값이기에, 제곱을 해버리면 값이 너무 커지면서 이상치에 더 민감해지기 때문이다.

```
plt.hist(train_mae_loss, bins=50, zorder=10)
plt.xlabel("Train MAE loss")
plt.ylabel("No of samples")
plt.title("Histogram of Training MAE Loss")
plt.grid(zorder=3)
plt.show()
```

재구성된 값과 원래의 값 사이의 오류를 히스토그램으로 나타낸다.

```
threshold = np.max(train_mae_loss)
plt.plot(x_train[0], label="Input Sequence")
plt.plot(x_train_pred[0], label="Reconstructed Sequence")
plt.title("Input vs Reconstructed Sequence")
plt.xlabel("Time Step")
plt.ylabel("Value")
plt.legend()
plt.grid()
plt.show()
```

threshold라는 값을 설정한다.
만약 오류가 이 값보다 크면 이상치로 간주한다.
코드에서는 threshold를 Training Loss의 최댓값으로 정의했다.

threshold를 정의한 후, 원본 시퀀스와 재구성된 시퀀스를 plot 한다.

02 실습 #3 코드 분석

```
df_test_value = (df_daily_jumpsup - training_mean) / training_std
fig, ax = plt.subplots()
df_test_value.plot(legend=False, ax=ax)
plt.title("Test Data")
plt.show()
```

Test Set (이상치 있는 데이터)를 나타낸다.

```
x_test_pred = model.predict(x_test)
test_mae_loss = np.mean(np.abs(x_test_pred - x_test), axis=1)
test_mae_loss = test_mae_loss.reshape((-1))
```

이번에는 Test Set에 대해 작업한다.

우선 모델에 데이터를 집어넣고, MAE를 정의한다.

```
plt.hist(test_mae_loss, bins=50, zorder=5)
plt.xlabel("test MAE loss")
plt.ylabel("No of samples")
plt.title("Histogram of Test MAE Loss")
plt.grid(zorder=3)
plt.show()
```

Test Set에 대해 MAE 값을 히스토그램으로 시각화한다.

02 실습 #3 코드 분석

```
anomalies = test_mae_loss > threshold

anomalous_data_indices = []
for data_idx in range(TIME_STEPS - 1, len(df_test_value) - TIME_STEPS + 1):
    if np.all(anomalies[data_idx - TIME_STEPS + 1 : data_idx]):
        anomalous_data_indices.append(data_idx)

df_subset = df_daily_jumpsup.iloc[anomalous_data_indices]
fig, ax = plt.subplots()
df_daily_jumpsup.plot(legend=False, ax=ax)
df_subset.plot(legend=False, ax=ax, color="r")
plt.title("Anomalies in Test Data")
plt.show()
```

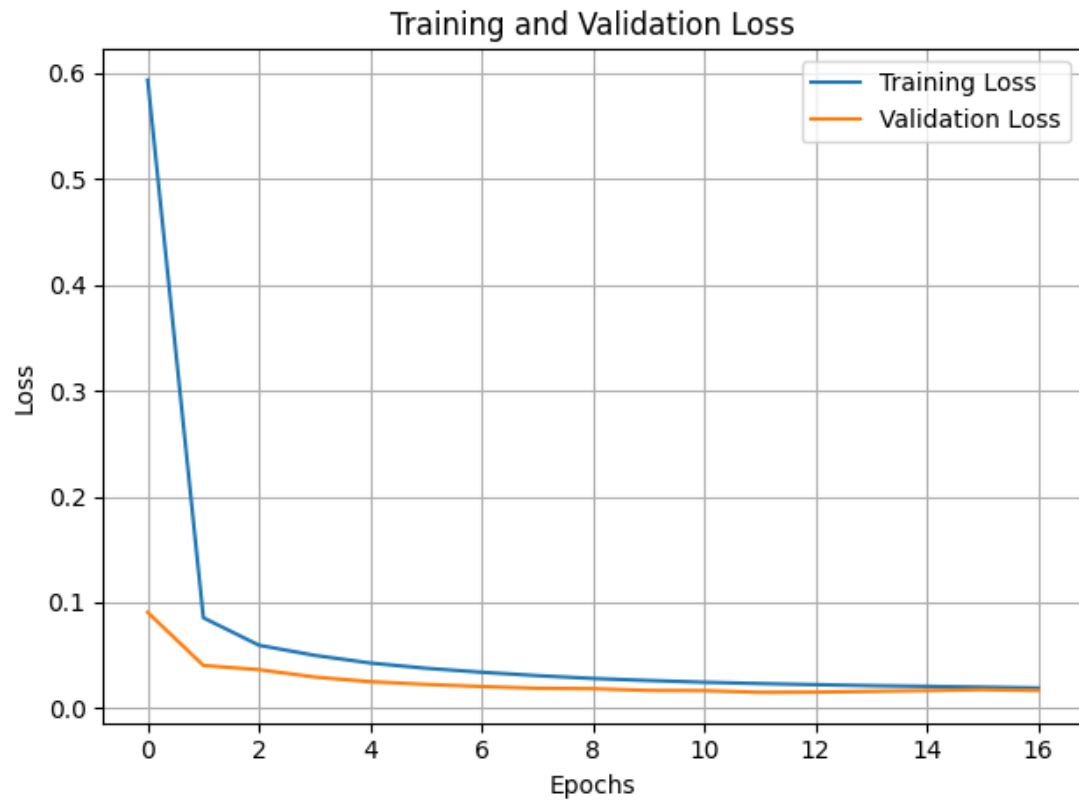
만약 오류가 threshold를 초과하는 경우 해당 데이터 포인트를 이상치로 간주한다.

그렇게 이상치로 판별된 데이터 포인트들의 인덱스를 찾고, 이를 빨간색으로 덧그린다.

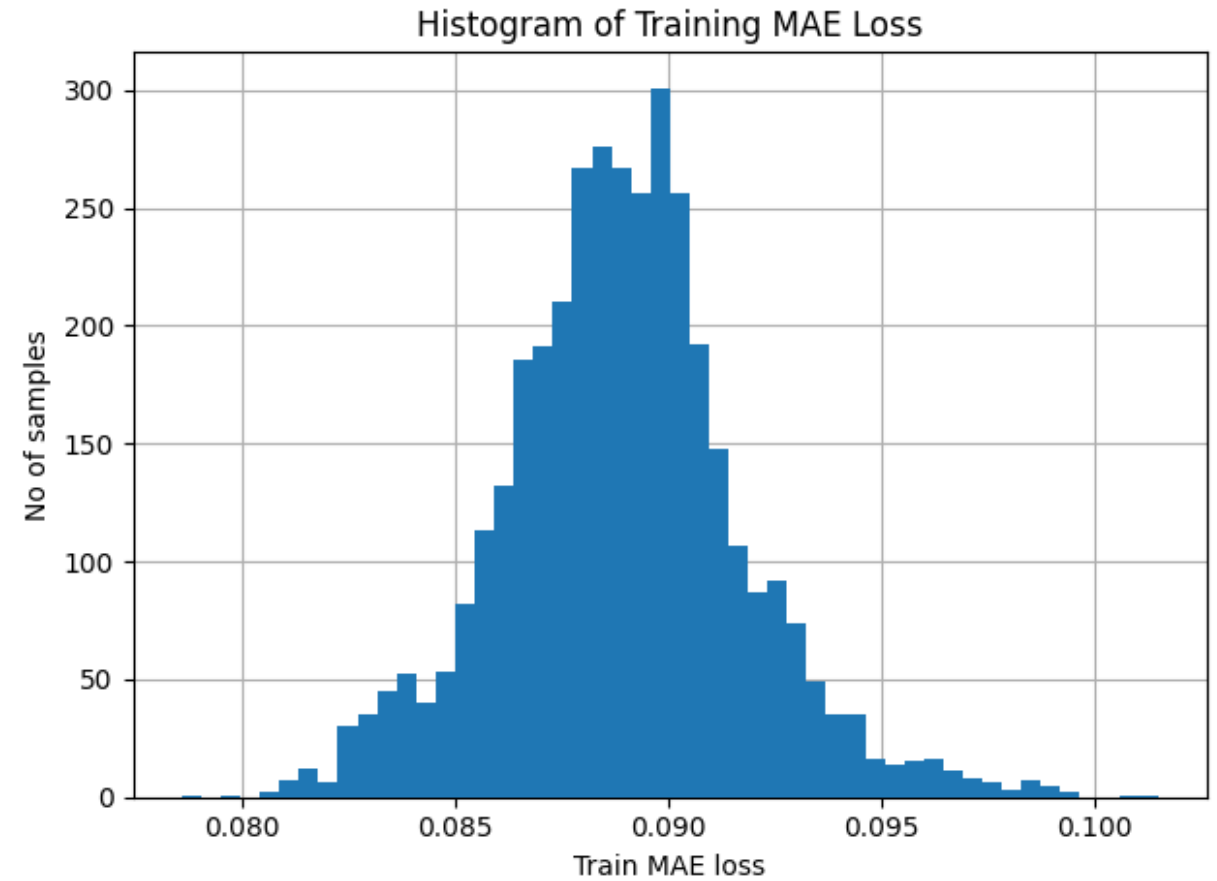
<코드의 그래프들 정리>

1. Training Loss, Validation Loss Plot
2. Training MAE Loss Histogram
3. Input Graph & Output Graph Plot
4. Test Set Graph Plot
5. MAE of Test Set Plot
6. Anomalies in Test Set Plot

02 실습 #3 실행 결과

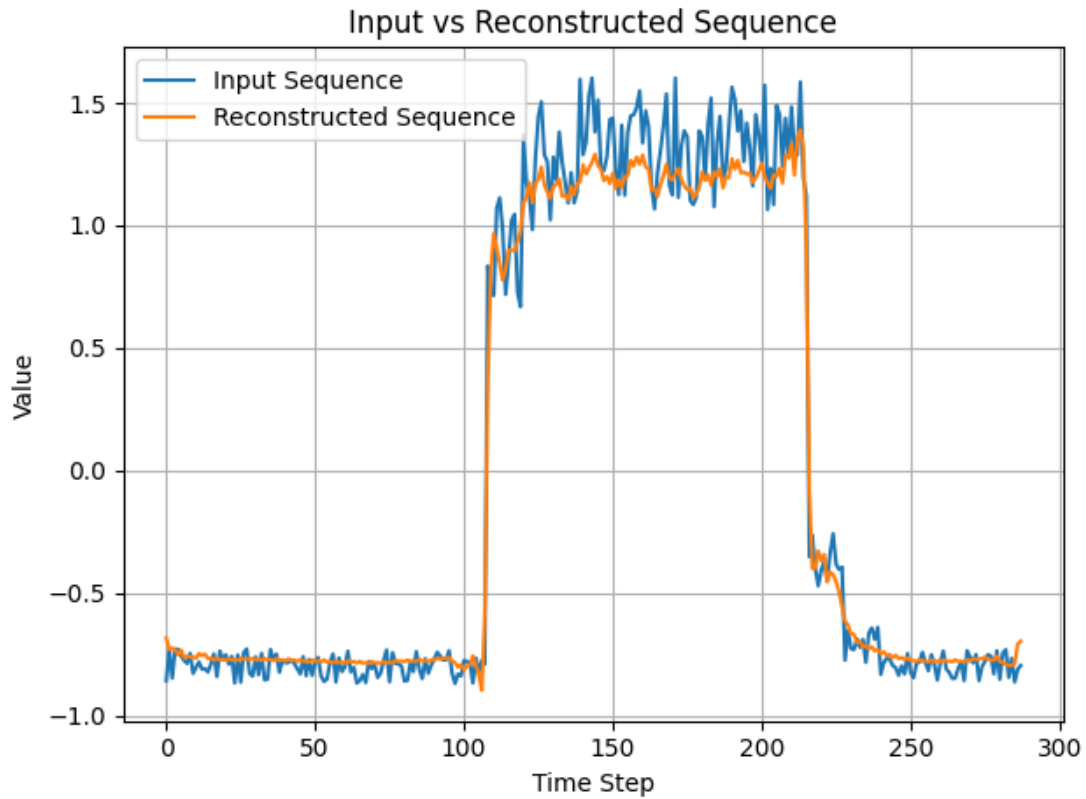


Early Stopping을 적용하였고, epoch이 17일 때 멈췄다.



원본 시퀀스와 모델로 재구성된 시퀀스 간의 MAE이다.

02 실습 #3 실행 결과

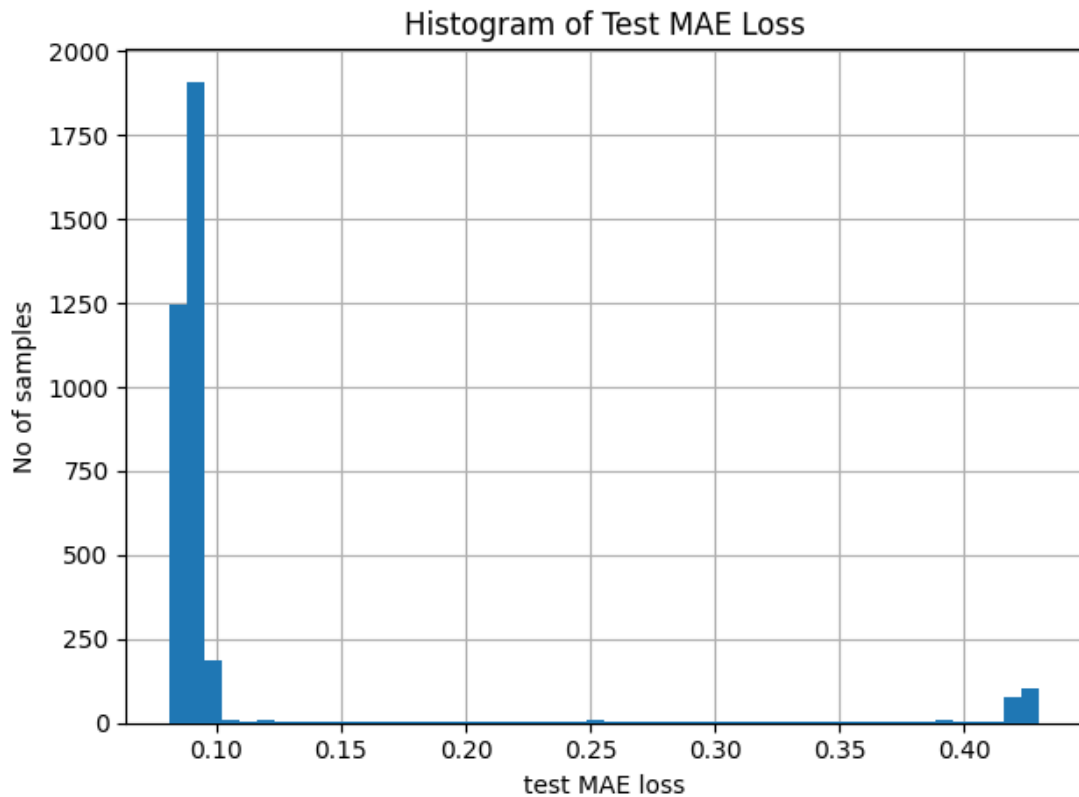


그래프를 보면, 전체적으로 비슷하게 따라간다는 걸 알 수 있다.

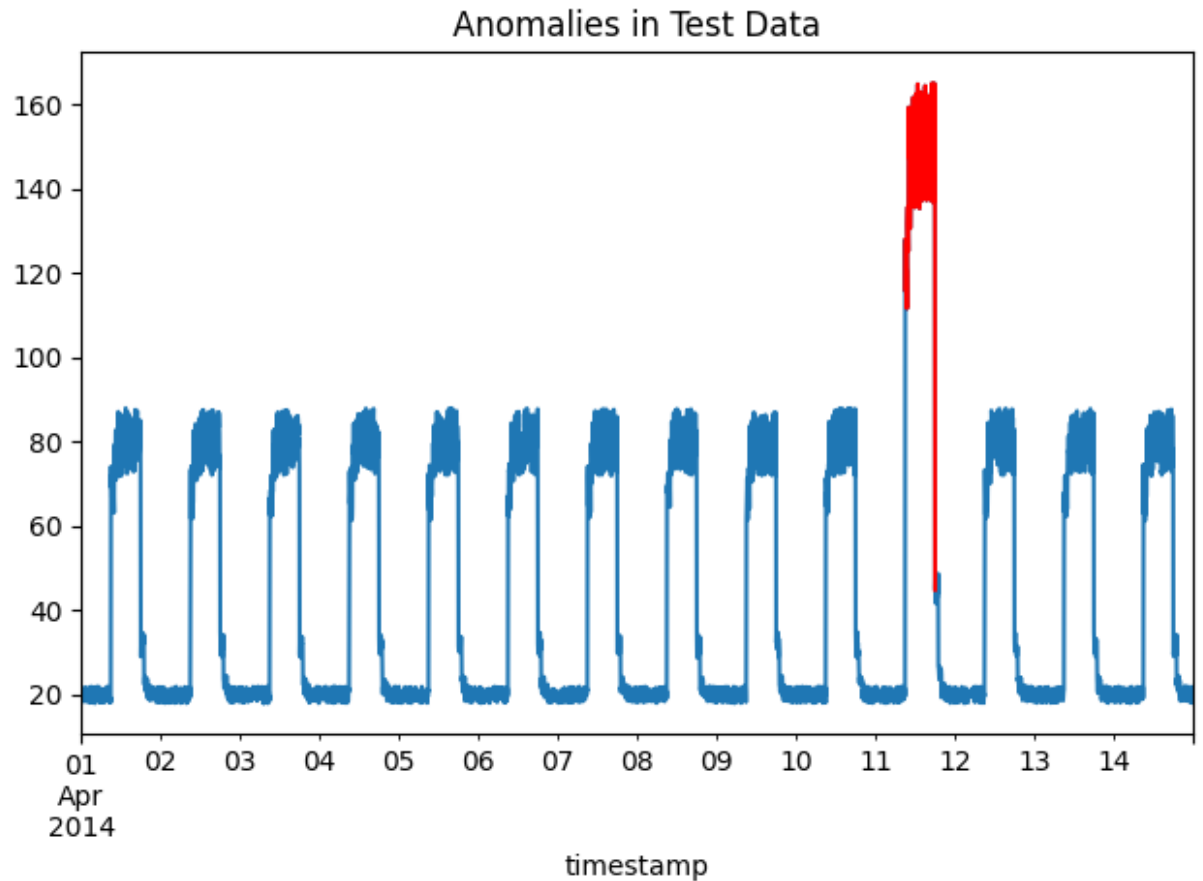


Test Set의 그래프이다.
저기 11과 12 사이에 있는 값들이 Outlier들이다.

02 실습 #3 실행 결과



Test Set의 Loss이다.
Outlier 부분을 제외하고는 Training Set과 유사하여
Loss가 그다지 크지 않은 것을 볼 수 있다.



Anomalies를 빨간색으로 덧칠한 Test Set이다.
예상대로 11과 12 사이의 값들이 이상치로 계산되어
빨간색으로 강조된 것을 볼 수 있다.

The background is a solid dark blue with several abstract geometric shapes. A large, light blue curved shape is on the right side. A dark blue diagonal band runs from the bottom left towards the top right. Another dark blue shape is in the top left corner.

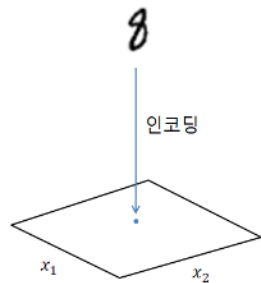
03

VAE

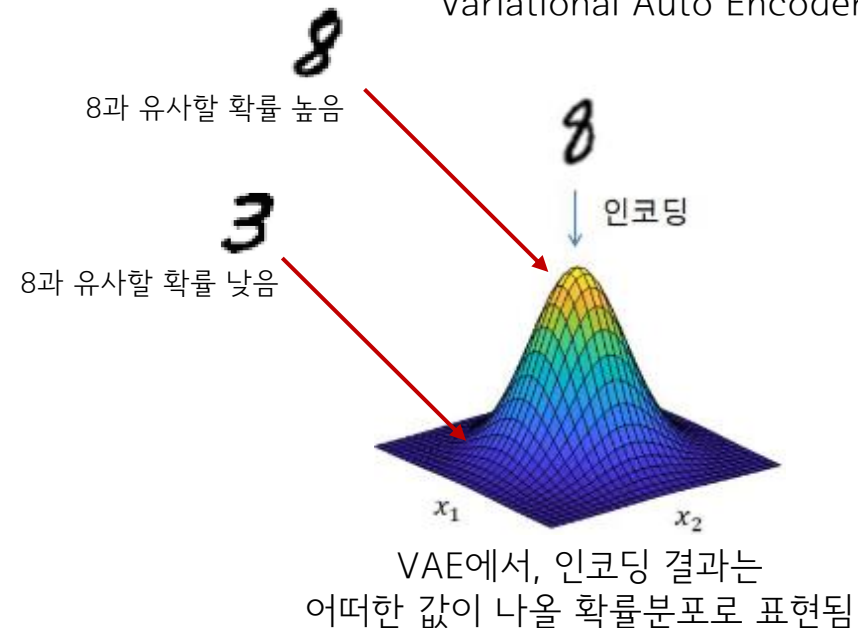
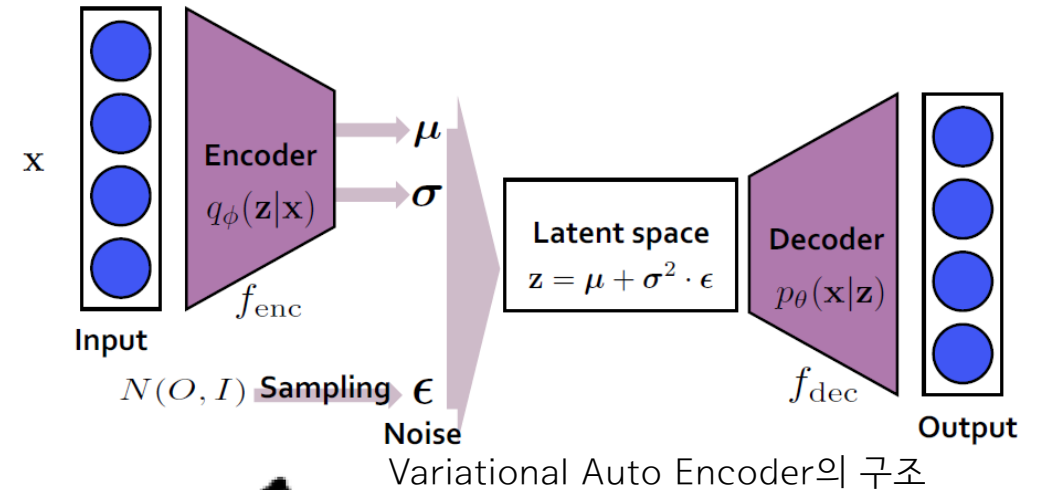
Variational Auto Encoder

VAE Variational Auto Encoder

- 주어진 데이터의 분포를 학습하여 새로운 데이터를 생성하는 데 사용
- 인코더는 입력을 Latent Space로 변환
이 때, 그냥 오토인코더와 다르게 Latent Vector z 의 확률분포를 추정
- 디코더는 Latent Space를 입력으로 변환
Latent Vector z 를 입력 데이터 x 로 변환



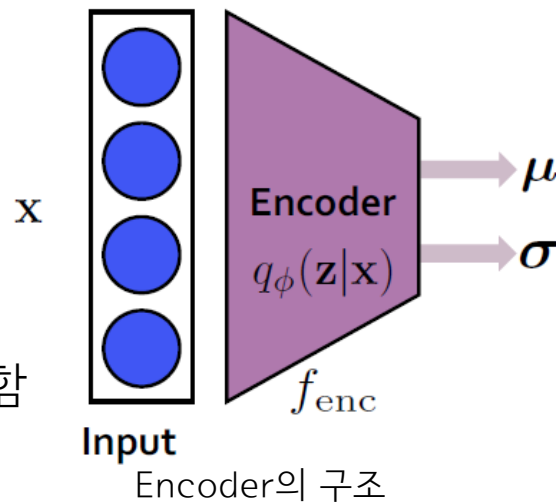
오토인코더에서, 인코딩 결과는 어떠한 값으로 표현됨



인코더

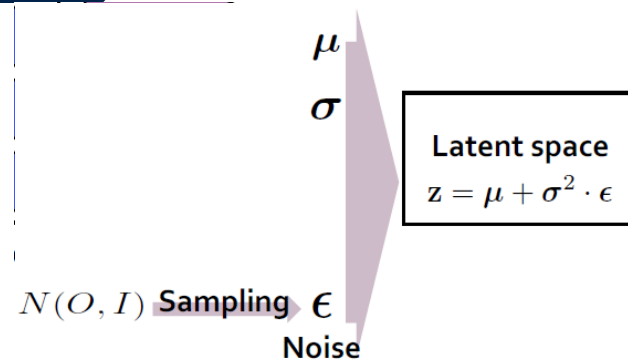
Variational Auto Encoder: Encoder

- Encoder는 input을 latent space로 변환하는 역할
- VAE의 인코더는 입력 x 가 주어졌을 때 Latent Vector z 의 분포를 나타내는 것을 목적으로 함
- Latent Vector z 는 입력 x 의 특징을 저차원에서 확률적으로 나타내는 분포
따라서, z 의 확률 분포의 가장 높은 점(Latent Vector z 는 PDF이다.)은 입력 x 와 동일하다 보기는 어려움
- $q(z|x)$ 를 approximate하는 것이 목적이므로, 이 분포를 나타내는
평균(μ)과 표준편차(σ)를 찾는 것이 목적이 됨



잠재 공간 Variational Auto Encoder: Latent Space

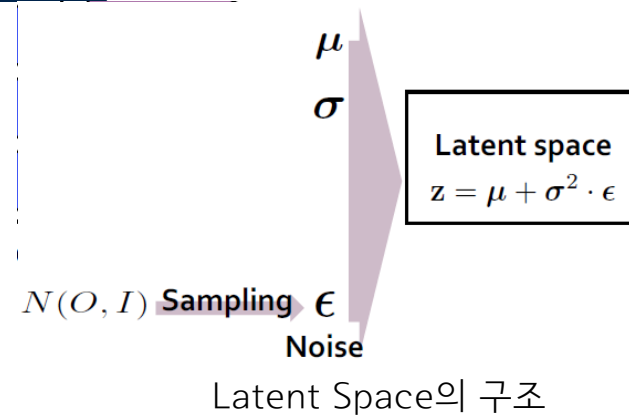
- 오토인코더와 동일하게, 잠재 공간은 인코더로부터 도출된 숨겨진 벡터들을 의미
- 오토인코더는 입력을 받아 최대한 비슷하게 출력을 생성하는 것이 목적
=> 입력과 출력이 무조건 같은 크기일 수 밖에 없음
- 잠재 공간의 각 차원은 독립적인 확률분포를 가지며, VAE의 디코더는 각 차원의 분포에서 동시에 샘플링 된 하나의 벡터를 입력받음
 1. 잠재 공간이 n개의 차원으로 구성될 때, 각각의 차원들은 독립적인 확률분포로 정의되며, 각각의 분포들은 μ, σ 로 정의됨
 2. 각 분포 당 한 값을 독립적으로 샘플링하여, n차원 벡터 z 를 구성
 3. 디코더는 이 Latent Vector z 를 기반으로 출력 생성
- VAE의 Latent Space에서는 입력과 출력이 무조건 동일한 사이즈이며, Sampling은 미분이 되지 않는(학습이 불가능한) 문제를 해결하기 위하여 인코더로부터 얻은 parameter(μ, σ)에 노이즈 추가



Latent Space의 구조

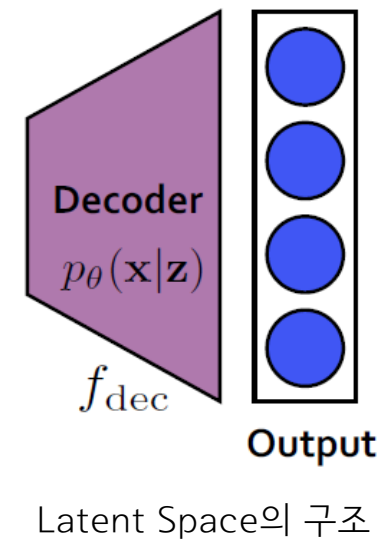
잠재 공간 Variational Auto Encoder: Latent Space

- 잠재 공간의 각 차원의 확률분포에 대해, 어떤 정규분포 (평균 0, 표준편차 1)로부터 하나의 noise를 무작위로 샘플링하여 얻은 후 이를 ϵ 이라 부름
- 이 ϵ 이라는 노이즈에 잠재 공간의 각 확률분포의 분산을 곱하고 평균을 더해서 latent vector z 를 얻음
- 이를 Reparametrization trick이라고 함
- 노이즈를 샘플링하는 것은 그냥 아무 점이나 찍는것이므로, Sampling은 수학적 연산으로 대체되었고, 따라서 미분이 가능해짐 = Back Propagation이 가능해짐 = 학습이 가능해짐
(이 때, 각 확률분포의 분산과 평균은 어느 정도 정규화가 되어 있어야 함)



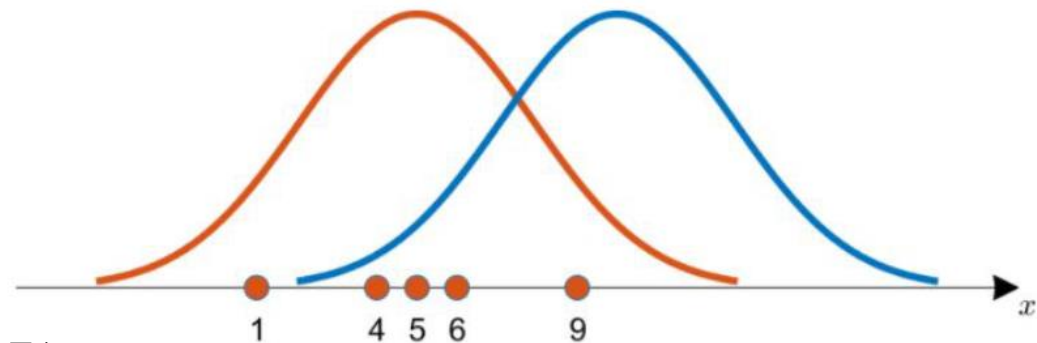
디코더 Variational Auto Encoder: Decoder

- Decoder는 encoder와 반대로 latent space를 input으로 변환하는 역할을 함
- 이 때, decode는 input의 true distribution, 달리 말하면 latent vector z 가 주어졌을 때 x 의 분포, 즉 위 그림에서 나타나듯이 $p(z|x)$ 를 추정하는 것을 목적으로 함
- 어떤 z 라는 vector가 주어짐에 따라 다시 데이터 x 를 generate하는 역할을 하기 때문에, decoder는 Generative Model의 역할을 수행할 수 있음



MLE Maximum Likelihood Estimation

- Likelihood는 쉽게 말해 두 분포의 평균과 표준편차가 얼마나 비슷한지, 즉 두 분포의 그래프가 얼마나 닮았는지에 대한 값
(또는, $P(Data|Parameter)$: 파라미터가 주어졌을 때 데이터가 나올 확률)
- 만약 데이터가 IID Condition(Independent and Identically Distribution), 즉 모든 데이터가 서로 독립이고, 각 데이터의 분포가 정규분포의 형태를 가진다면



주황 분포: 실제 분포
파랑 분포: 예측된 파라미터로 그린 분포
파랑 분포가 주황 분포와 비슷해지려면 Likelihood가 최대가 되어야 한다.

- $P(x_i|\mu, \delta^2) = \frac{1}{\sqrt{2\pi\delta^2}} e^{-\frac{(x_i-\mu)^2}{2\delta^2}}$, $Likelihood = \prod_{i=1}^n p(x_i|\mu, \delta^2)$ 이며, (n은 데이터의 개수)
우리의 목적은 두 그래프가 겹치는 것 (동일해지는 것)이므로 Likelihood는 최대가 되어야 한다.
- $\prod$ 를 없애기 위해 $\log$ 를 취하면 $\prod$ 는 $\sum$ 로 변환됨
($\log$ 는 단조증가 함수이기 때문에 씌워도 최대를 구하는 데 문제 없음)
- $Log Likelihood = \sum_{i=1}^n \left(\log \frac{1}{\sqrt{2\pi\delta^2}} - \frac{(x_i-\mu)^2}{2\delta^2} \right)$ 로 정의된다.

KL Divergence

Kullback-Leibler Divergence

- 두 확률 분포 간의 차이를 측정하는 비대칭적인 척도
- 한 분포를 기준으로 다른 분포가 얼마나 다른지를 정량적으로 나타냄
- Relative Entropy라고도 함

- 이산적 KL Divergence에서,
$$\mathbb{KL}(p||q) = \sum_{k=1}^K p_k \log p_k - \sum_{k=1}^K p_k \log q_k$$

 $\Rightarrow \mathbb{KL}(p||q) = (-\text{entropy of } p) - (\text{cross entropy of } p \text{ \& } q)$ 이다.

$$\mathbb{KL}(p||q) \triangleq \sum_{k=1}^K p_k \log \frac{p_k}{q_k}$$

확률 분포가 이산적일때의
KL Divergence

$$\mathbb{KL}(p||q) \triangleq \int dx p(x) \log \frac{p(x)}{q(x)}$$

확률 분포가 연속적일때의
KL Divergence

- p를 실제 분포, q를 우리가 추정한 분포라고 가정하면, 위 식을 실제 분포 p와 비교했을 때 q를 이용해 p로부터 나온 데이터를 표현할 때 얼마나 많은 정보량(=bit)이 추가적으로 사용되어야 하는지를 표시한 것이라고 생각할 수 있음
즉, 이 값이 낮으면 추정한 분포 q는 실제 분포 p와 비슷한 복잡도를 가지기 때문에 p와 비슷한 분포이고, 이 값이 높을수록 추가적으로 많은 정보량이 더 소모되어야 하므로 p와 다른 분포임

ELBO Evidence Lower Bound

- VAE는 MLE를 이용하기 때문에, 기존의 MSE나 CEE같은 오차로는 한계가 있음
- 목표: 주어진 입력에 대한 Log Likelihood($\log p(x)$)를 최대화
=> 그러나, $\log p(x)$ 를 직접적으로 계산하기 어려운 경우가 많음
=> 대신 $\log p(x)$ 의 하한을 설정 후, 이를 최대화
- Reconstruction Loss: 입력 x 와 디코더를 통해 출력된 x' 의 차이 측정 (MSE나 CEE 사용)
- Regularization Loss: 인코더에서 추정한 잠재 변수 z 의 분포 $q(z|x)$ 와 prior distribution $p(z)$ 간의 Kullback-Leibler 발산(KL Divergence)으로 정의 (잠재 변수의 분포가 prior distribution과 얼마나 다른지를 나타냄)

$$\begin{aligned}\log p_{\theta}(x) &= \int q_{\phi}(z|x) \log p_{\theta}(x) dz && \text{Bayes' Rule 사용} \\ &= \int q_{\phi}(z|x) \log \frac{p_{\theta}(x|z)p(z)}{p_{\theta}(z|x)} dz \\ &= \int q_{\phi}(z|x) \log \frac{p_{\theta}(x|z)p(z)}{p_{\theta}(z|x)} \frac{q_{\phi}(z|x)}{q_{\phi}(z|x)} dz && \text{KL Divergence 사용} \\ &= \int q_{\phi}(z|x) \log p_{\theta}(x|z) dz - KL(q_{\phi}(z|x)||p(z)) + KL(q_{\phi}(z|x)||p_{\theta}(z|x)) \\ &= \int q_{\phi}(z|x) \log p_{\theta}(x|z) dz - KL(q_{\phi}(z|x)||p(z)) + KL(q_{\phi}(z|x)||p_{\theta}(z|x)) \\ &\geq \int q_{\phi}(z|x) \log p_{\theta}(x|z) dz - KL(q_{\phi}(z|x)||p(z)) \leftarrow \text{Evidence lower bound (ELBO)} \\ &\quad \text{Reconstruction error} \quad \text{Regularization error}\end{aligned}$$

ELBO의 유도 과정

실습 #4) VAE를 활용한 MNIST 데이터 생성

- 앞선 실습을 따라하여 각 Latent vector에 따른 숫자 데이터 생성
- 코드 내용이 복잡하고 어렵기 때문에, 파이썬에 대한 추가적인 공부 요망
ex) Class, 디코레이터 등등
- 잠재벡터를 -2~2 사이에서 총 900개의 숫자 데이터 생성
- VAE 구조 및 파라미터 조정에 따른 결과 비교 분석
- 각 잠재벡터 마다 알맞은 숫자가 생성되었는지 확인 -> 다음 페이지 참고
- 단순 과제 제출 목적이 아닌, VAE에 대한 이해를 목적으로 진행 요망

03 실습 #4 코드 분석

```
import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
import tensorflow as tf
import tensorflow.keras as keras
from tensorflow.keras import layers

class Sampling(layers.Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        batch = tf.shape(z_mean)[0]
        dim = tf.shape(z_mean)[1]
        epsilon = tf.random.normal(shape=(batch, dim))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon
```

필요한 라이브러리를 임포트한다.

Keras의 layers.Layer를 상속받아 사용자 정의 레이어를 만든다.

이 클래스는 잠재 공간에서 샘플을 생성하는 역할을 한다.

Call()은 잠재 공간의 파라미터 (z_mean, z_log_var)를 사용하여 랜덤 샘플링(epsilon)을 수행하며, 이는 VAE에서 잠재 공간의 다양한 표현을 얻는 데 사용된다.

03 실습 #4 코드 분석

```
class VAE(keras.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
        self.total_loss_tracker = keras.metrics.Mean(name="total_loss")
        self.reconstruction_loss_tracker = keras.metrics.Mean(name="reconstruction_loss")
        self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")

    @property
    def metrics(self):
        return [
            self.total_loss_tracker,
            self.reconstruction_loss_tracker,
            self.kl_loss_tracker
        ]

    def train_step(self, data):
        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = self.encoder(data)
            reconstruction = self.decoder(z)
            reconstruction_loss = tf.reduce_mean(
                tf.reduce_sum(keras.losses.binary_crossentropy(data, reconstruction), axis=(1, 2))
            )
            kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var))
            kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))
            total_loss = reconstruction_loss + kl_loss

        grads = tape.gradient(total_loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
        self.total_loss_tracker.update_state(total_loss)
        self.reconstruction_loss_tracker.update_state(reconstruction_loss)
        self.kl_loss_tracker.update_state(kl_loss)

        return {
            "loss": self.total_loss_tracker.result(),
            "reconstruction_loss": self.reconstruction_loss_tracker.result(),
            "kl_loss": self.kl_loss_tracker.result()
        }
```

VAE 클래스를 정의한다.

Reconstruction loss와 KL loss를 계산하고, 이 두 loss의 합을 total loss로 사용한다.

train_step()에서는 모델의 한 학습 단계를 정의한다.

tf.GradientTape()를 사용하여 loss에 대한 가중치의 gradient를 계산한다.

VAE 클래스의 metrics 메서드에 사용된 @property 데코레이터는 메서드를 속성처럼 사용할 수 있도록 만들어주는 기능이다.

metrics 메서드에 @property라는 데코레이터를 사용하여 Keras가 모델의 컴파일 및 학습 과정에서 자동으로 metric을 추적하고 보고할 수 있도록 하기 위하여 추가했다.

metrics의 속성으로 total_loss_tracker, reconstruction_loss_tracker, kl_loss_tracker와 같은 loss metric을 정의하여,

학습 과정에서 loss와 관련된 값을 매 epoch마다 keras가 자동으로 추적하고 기록할 수 있게 되었다.

03 실습 #4 코드 분석

```
def plot_combined(vae, data, labels, n=30, figsize=15):
    digit_size = 28
    scale = 2.0
    figure = np.zeros((digit_size * n, digit_size * n))
    grid_x = np.linspace(-scale, scale, n)
    grid_y = np.linspace(-scale, scale, n)[::-1]
    for i, yi in enumerate(grid_y):
        for j, xi in enumerate(grid_x):
            z_sample = np.array([[xi, yi]])
            x_decoded = vae.decoder.predict(z_sample)
            digit = x_decoded[0].reshape(digit_size, digit_size)
            figure[
                i * digit_size : (i + 1) * digit_size,
                j * digit_size : (j + 1) * digit_size
            ] = digit
    z_mean, _, _ = vae.encoder.predict(data)
    # Latent Space
    plt.subplot(1, 2, 1)
    start_range = digit_size // 2
    end_range = n * digit_size + start_range
    pixel_range = np.arange(start_range, end_range, digit_size)
    sample_range_x = np.round(grid_x, 1)
    sample_range_y = np.round(grid_y, 1)
    plt.xticks(pixel_range, sample_range_x)
    plt.yticks(pixel_range, sample_range_y)
    plt.xlabel("z[0]")
    plt.ylabel("z[1]")
    plt.imshow(figure, cmap="Greys_r")
    plt.title("Latent Space")
    # Label
    plt.subplot(1, 2, 2)
    plt.scatter(z_mean[:, 0], z_mean[:, 1], c=labels, s=3, cmap="tab10")
    plt.colorbar()
    plt.xlabel("z[0]")
    plt.ylabel("z[1]")
    plt.title("Label Clusters")
    plt.grid()
    plt.show()
```

잠재 공간에서 다양한 잠재 벡터(z_sample)를 샘플링하고, 이를 디코더에 입력하여 이미지를 복원한 후, 이를 시각화하는 함수이다.

n 값에 따라 $n \times n$ 개수의 이미지를 생성하여 하나의 큰 이미지($figure$)로 구성하며, $scatter$ 를 통해 비슷한 데이터가 잠재 공간에서 가깝게 위치하는지를 확인할 수 있다.

GPU 가속을 위하여 현재 Tensorflow 2.6.0 버전을 사용중이기에, 학습 자료의 함수에서 버전 이슈가 발생하여 부득이하게 두 함수를 하나로 합쳤다.

03 실습 #4 코드 분석

```
latent_dim = 2
encoder_inputs = tf.keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(32, 3, activation='relu', strides=2, padding='same')(encoder_inputs)
x = layers.Conv2D(64, 3, activation='relu', strides=2, padding='same')(x)
x = layers.Flatten()(x)
x = layers.Dense(16, activation='relu')(x)
z_mean = layers.Dense(latent_dim, name="z_mean")(x)
z_log_var = layers.Dense(latent_dim, name="z_log_var")(x)
z = Sampling()([z_mean, z_log_var])
encoder = tf.keras.Model(encoder_inputs, [z_mean, z_log_var, z], name="encoder")
```

잠재 변수의 차원을 2로 설정한 다음, MNIST의 크기인 28 by 28의 흑백 이미지를 입력으로 받는다.
특징 추출을 위해 2개의 Convolution 레이어를 거친 다음, Dense에서 z_mean과 z_log_var를 계산한다.
Sampling을 이용하여 잠재 변수 z를 샘플링하고, 마지막으로 encoder 모델에서 z_mean, z_log_var, z를 출력한다.

```
latent_inputs = tf.keras.Input(shape=(latent_dim,))
x = layers.Dense(7 * 7 * 64, activation='relu')(latent_inputs)
x = layers.Reshape((7, 7, 64))(x)
x = layers.Conv2DTranspose(64, 4, activation='relu', strides=2, padding='same')(x)
x = layers.Conv2DTranspose(32, 4, activation='relu', strides=2, padding='same')(x)
decoder_outputs = layers.Conv2DTranspose(1, 3, activation='sigmoid', padding='same')(x)
decoder = keras.Model(latent_inputs, decoder_outputs, name="decoder")
```

Dense 레이어에서 잠재 변수를 큰 형태로 변환하고, 이후 이미지 복원을 시작한다.
Reshape 레이어에서 형태 변환 후, Transposed Convolution 연산을 통하여 원래 이미지로 복원하려는 연산을 수행한다.
마지막으로 출력 레이어에서는 sigmoid를 활용하여 픽셀 값을 0~1로 제한하고, decoder 모델에서는 z를 이용하여 복원된 이미지를 출력하게 된다.

03 실습 #4 코드 분석

```
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
mnist_digits = np.concatenate([x_train, x_test], axis=0)
mnist_digits = np.expand_dims(mnist_digits, -1).astype("float32") / 255
```

MNIST 데이터셋을 불러온다. 이렇게 불러온 데이터는 모델의 학습 용도로 쓰이게 된다.
전처리로 (28, 28)을 (28, 28, 1)로 변환한 후, 픽셀의 값을 0~1로 정규화하여 모델의 학습을 돕는다.

```
vae = VAE(encoder, decoder)
vae.compile(optimizer=tf.keras.optimizers.Adam())
vae.fit(mnist_digits, epochs=50, batch_size=128)
```

인코더와 디코더를 조합하여 VAE 모델을 생성하고, adam을 옵티마이저로 한다.
이후, 50 epoch 동안 학습을 진행한다. (강의 자료보다 살짝 늘렸다.)

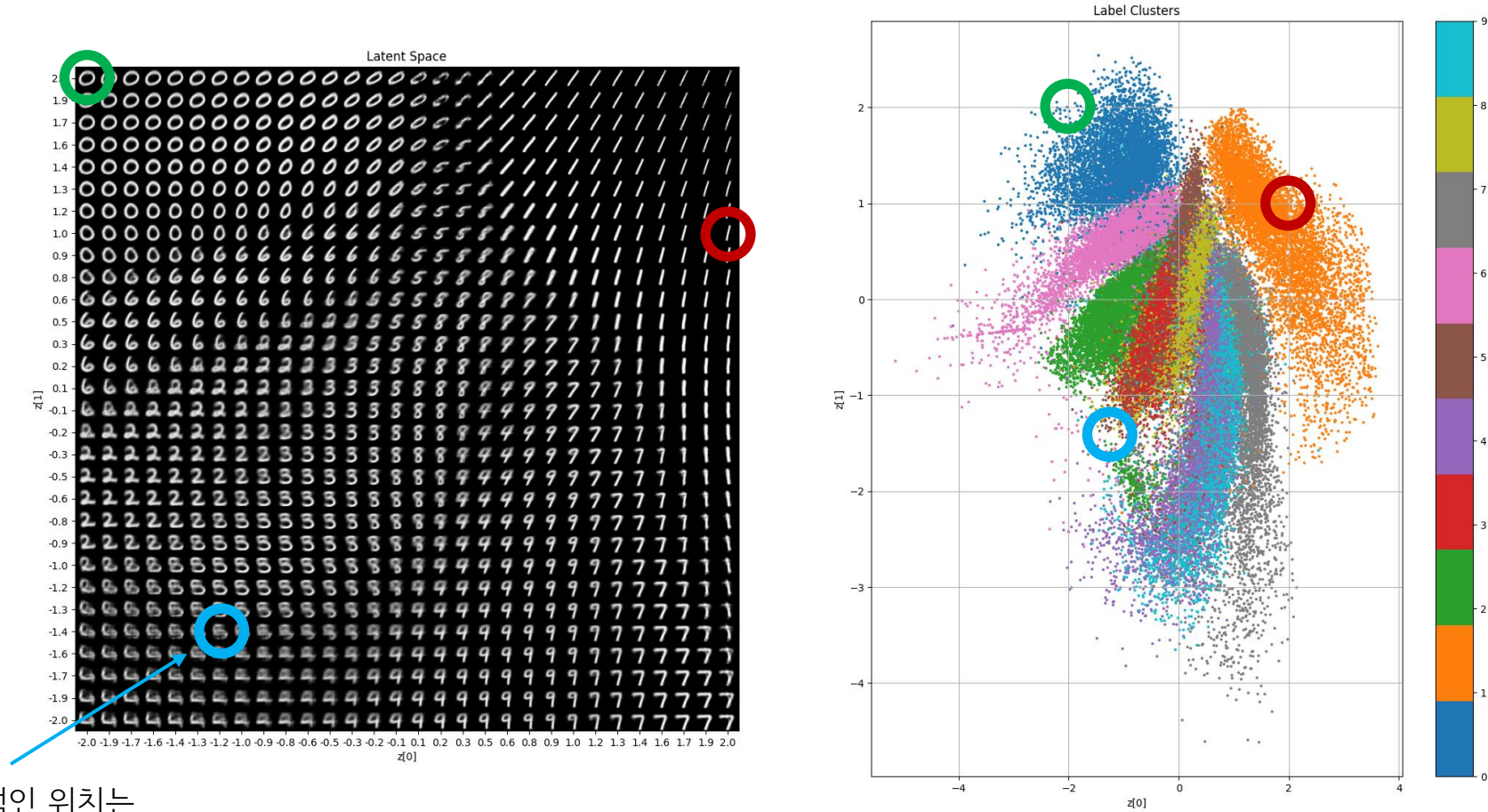
```
(x_train, y_train), _ = tf.keras.datasets.mnist.load_data()
x_train = np.expand_dims(x_train, -1).astype("float32") / 255
```

MNIST 데이터셋을 한번 더 불러온다.
이번에 불러온 데이터는 라벨 정보를 이용하여 시각화에 쓰이게 된다.

```
plot_combined(vae, x_train, y_train)
```

잠재 벡터별 생성값과 군집을 표시한다.

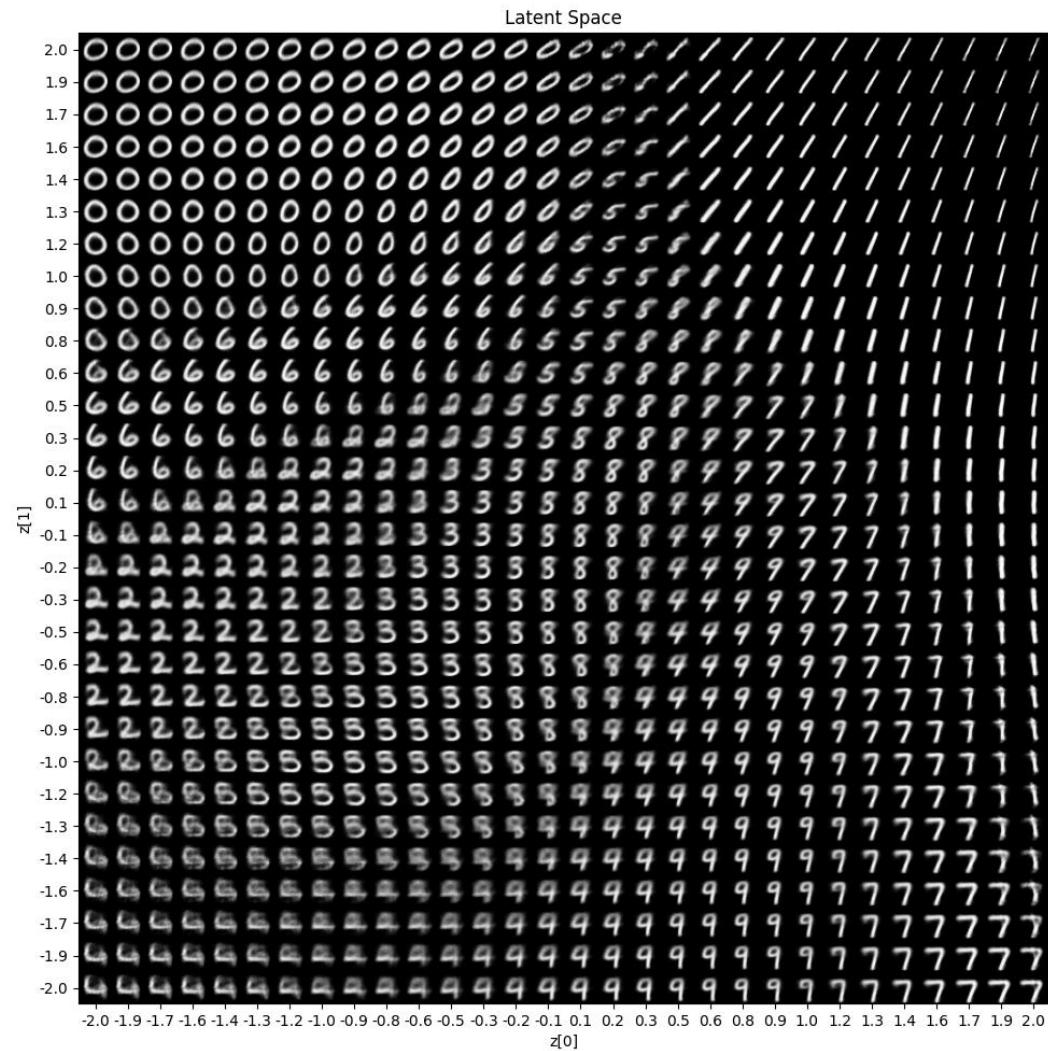
Variational Auto Encoder



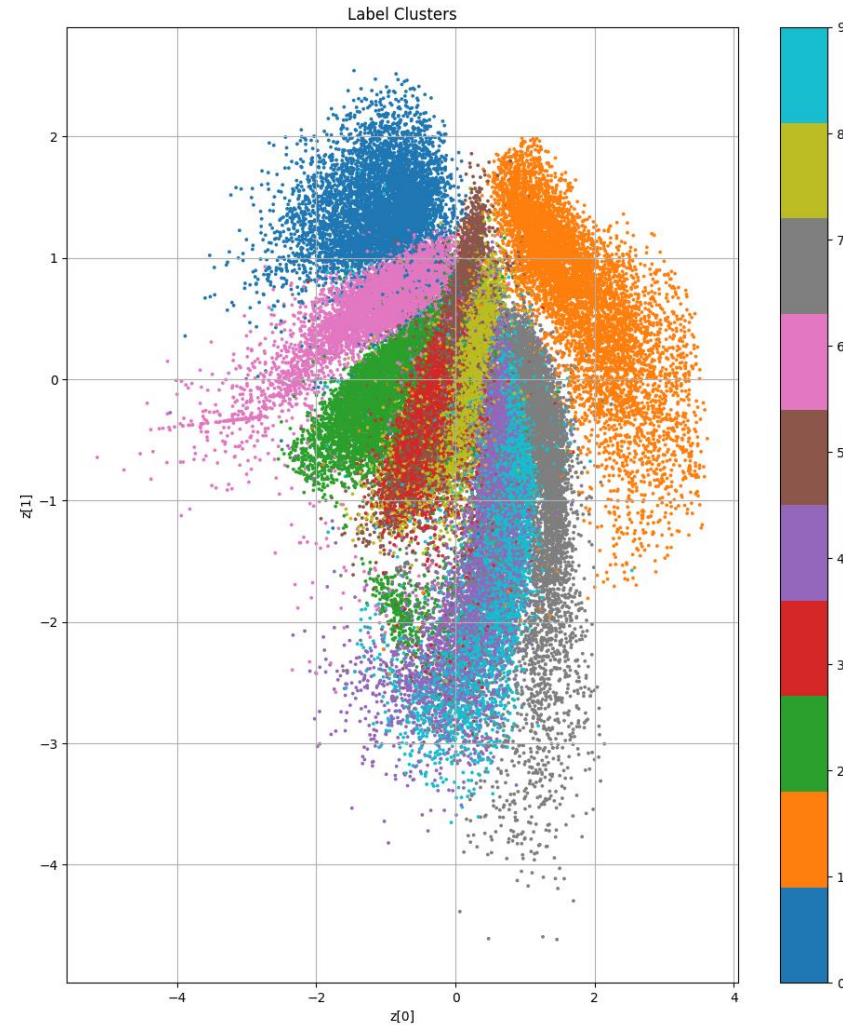
여러 군집이 섞인 위치는
생성값 또한 숫자들이
섞인 형태인 것을 볼 수 있다.

cmap으로 tab10을 사용하여 각 점들이 잘 구분되도록 하였다.
Latent Clusters에서 (2, 1) 부근은 주황색, 즉 1인걸 알 수 있고,
해당 좌표의 Latent Space 역시 1로 생성된 것을 알 수 있다.

Latent Vector Visualization



Latent Vector Cluster Visualization



고급머신러닝 보고서
감사합니다